

סיכום מקיף במבני נתונים ואלגוריתמים

סיכום מקיף במבני נתונים ואלגוריתמים

תוכן עניינים

- 1. ניתוח יעילות וסיבוכיות אלגוריתמים
 - [מבוא והגדרות יסוד](#)
 - [טבלת סדרי גודל נפוצים \(בסדר עולה\)](#)
 - [סימונים אסימפטוטיים](#)
 - [תכונות של חסמים אסימפטוטיים](#)
 - [טורים וסדרות ידועים](#)
 - [שלבי עבודה מעשיים לניתוח סיבוכיות](#)
 - [משפט הסנדוויץ' לחישוב חסמים אסימפטוטיים הדוקים](#)
- 2. משוואות רקורסיה ואלגוריתמים רקורסיביים
 - [שיטת העץ \(Iteration Tree Method\)](#)
 - [שיטת ההצבה \(Substitution Method\)](#)
 - [שיטת המאסטר \(Master Theorem\)](#)
 - [שיטת החלפת משתנים \(Change of Variables\)](#)
- 3. ניתוח לשיעורין (Amortized Analysis)
 - 1. [שיטת הצבירה \(Aggregate Method\)](#)
 - 2. [שיטת החיובים / הבנקאי \(Accounting Method\)](#)
 - 3. [שיטת הפוטנציאל \(Potential Method\)](#)
 - [דוגמאות ויישומים מעשיים](#)
 - [דה-אמורטיזציה \(Deamortization\)](#)
- 4. מבני נתונים בסיסיים ורשימות מקושרות
 - [טיפוס נתונים אבסטרקטי \(ADT\) מול מימוש פיזי](#)
 - [אלגוריתמי חיפוש במערך ממין](#)
 - [בעיות סכום: Sum-2 ו-Sum-3](#)
 - [רשימות מקושרות](#)
 - [אלגוריתמים מתקדמים ברשימות מקושרות](#)
- 5. מחסניות ותורים
 - [מחסנית \(Stack\)](#)
 - [תור \(Queue\)](#)
 - [מימוש תור באמצעות שתי מחסניות \(Two-Stack Queue\)](#)
- 6. מבוא לגרפים
 - [הגדרות פורמליות](#)
 - [דרכי ייצוג גרפים בזיכרון](#)
- 7. עצים ועצי חיפוש בינאריים (BST)
 - [מושגי יסוד בעצים](#)

- [עץ בינארי](#)
- [סריקות בעצים בינאריים](#)
- [שחזור עץ בינארי מסריקות](#)
- [עץ חיפוש בינארי \(BST - Binary Search Tree\)](#)
- 8. [עצי חיפוש מאוזנים \(AVL Trees\)](#)
 - [משפט: חסם הגובה בעץ AVL](#)
 - [פעולות איזון ותיקון בעקבות שינויים](#)
 - [שימושים מיוחדים וחישובים מתקדמים ב-AVL](#)
- 9. [עצי חיפוש מסדר גבוה \(B-Trees\)](#)
 - [פעולות ב-B-Tree](#)
- 10. [ערימות \(Heaps\)](#)
 - [פעולות בערימה בינארית](#)
 - [בניית ערימה \(Build-Heap\) בזמן ליניארי \$O\(n\)\$](#)
 - [ערימת חציון \(Median Heap\)](#)
 - [ערימה בינומית \(Binomial Heap\)](#)
- 11. [טבלאות גיבוב \(Hash Tables\)](#)
 - [מנגנוני גיבוב דטרמיניסטיים](#)
 - [מנגנונים אקראיים ומשפחות אוניברסליות](#)
 - [גיבוב מושלם דו-שלבי \(Perfect Hashing\)](#)
 - [טיפול בהתנגשויות](#)
- 12. [קבוצות זרות \(Union-Find / Disjoint Sets\)](#)
 - [דרכי מימוש ויעילות](#)
- 13. [פרדיגמות עיצוב אלגוריתמים](#)
 - [הפרד ומשול \(Divide & Conquer\)](#)
- 14. [מיונים ובעיות בחירה](#)
 - [חסם תחתון למיונים מבוססי השוואה](#)
 - [יציבות מיונים \(Stable Sorting\)](#)
 - [מיונים שאינם מבוססי השוואה](#)
 - [בעיית הבחירה \(Selection\)](#)
 - [יישום: מציאת \$k\$ האיברים הקרובים ביותר לחציון ב- \$O\(n\)\$](#)
- 15. [תכנון דינמי \(Dynamic Programming\)](#)
 - 15.1 [מבוא והגדרות יסוד](#)
 - 15.2 [מספרי פיבונאצ'י \(Fibonacci\)](#)
 - 15.3 [תת-סדרה הבלתי תלויה המקסימלית במערך \(MIS in Path\)](#)
 - 15.4 [תזמון מקטעי ממושקלים \(Weighted Interval Scheduling\)](#)
 - 15.5 [תת-סדרה משותפת ארוכה ביותר \(LCS\)](#)
 - 15.6 [כפל שרשרת מטריצות \(Matrix Chain Multiplication - MCM\)](#)
 - 15.7 [בעיית תרמיל הגב בשלמים \(Integer Knapsack\)](#)
 - 15.8 [קבוצה בלתי תלויה מקסימלית בעץ \(MIS in a Tree\)](#)
 - 15.9 [תת-סדרה עולה ארוכה ביותר \(LIS\)](#)
 - 15.10 [בעיית המזכירות / חלוקת סטודנטים לקבוצות](#)

1. ניתוח יעילות וסיבוכיות אלגוריתמים

מבוא והגדרות יסוד

אלגוריתם

אלגוריתם הוא סדרת צעדים מוגדרת היטב, המקבלת קלט כלשהו ומפיקה פלט רצוי בזמן סופי. אלגוריתם תקין חייב להחזיר את הערכים הנכונים עבור כל קלט תקין, ולהחזיר שגיאה או להתריע עבור קלט לא תקין.

כלי ההוכחה המרכזיים לנכונות של אלגוריתמים הם **אינדוקציה מתמטית** (על גודל הקלט או מספר האיטרציות) ו**הוכחה בשלילה**.

הערכת יעילות של אלגוריתם מתבצעת על ידי ניתוח צריכת המשאבים שלו ביחס לגודל הקלט n :

- סיבוכיות זמן (Time Complexity):** פונקציה המחשבת את מספר צעדי החישוב הבסיסיים (פעולות פרימיטיביות כגון השמה, השוואה, פעולה אריתמטית) כפונקציה של גודל הקלט n .
- סיבוכיות מקום (Space Complexity):** פונקציה המחשבת את כמות הזיכרון הדינמי או הנוסף הנדרש לאלגוריתם מעבר לזיכרון הקלט.

נבחין בין שלושה מקרי קצה בניתוח יעילות:

- המקרה הטוב (Best Case):** זמן הריצה הקצר ביותר של האלגוריתם על קלט כלשהו בגודל n .
- המקרה הגרוע (Worst Case):** זמן הריצה הארוך ביותר של האלגוריתם על קלט כלשהו בגודל n . זהו מדד היעילות החשוב ביותר כיוון שהוא מעניק חסם עליון מובטח לביצועים.
- המקרה הממוצע (Average Case):** תוחלת זמן הריצה על פני התפלגות מסוימת של כל הקלטים האפשריים בגודל n . לעיתים המושג מטעה, שכן ייתכן שאין אף קלט יחיד שזמן הריצה עליו שווה בדיוק לממוצע.

בניתוח אסימפטוטי מתעניינים רק ב**סדר הגודל** של זמן הריצה כאשר $n \rightarrow \infty$. מתעלמים מקבועים אדיטיביים ומולטיפליקטיביים הנגזרים מהמימוש בשפת המכונה או החומרה הספציפית, ומתמקדים באיבר הדומיננטי ביותר.

טבלת סדרי גודל נפוצים (בסדר עולה)

שם	סימון אסימפטוטי
קבוע	1
לוגריתמי	$\log n$
שורש	$\sqrt{n}$
ליניארי	n
ליניארי-לוגריתמי	$n \log n$
פולינומי	n^d (עבור $d > 0$)
מעריכי	2^n
עצרת / סופר-מעריכי	$n!$ או n^n

סימונים אסימפטוטיים

יהיו $f(n), g(n)$ שתי פונקציות חיוביות ממש מהטבעיים לממשיים החיוביים $(f, g : \mathbb{N} \rightarrow \mathbb{R}^+)$.

✓ הגדרה: חסם עליון אסימפטוטי (O)

נגדיר $f(n) \in O(g(n))$ אם ורק אם קיימים קבועים $c > 0$ ו- $n_0 \geq 0$ כך שלכל $n \geq n_0$ מתקיים:

$$f(n) \leq c \cdot g(n)$$

✓ הגדרה: חסם עליון אסימפטוטי קטן (o)

נגדיר $f(n) \in o(g(n))$ אם ורק אם לכל קבוע $c > 0$ קיים $n_0 \geq 0$ כך שלכל $n \geq n_0$ מתקיים $f(n) < c \cdot g(n)$. לחילופין, באמצעות גבולות:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

✓ הגדרה: חסם תחתון אסימפטוטי (Ω)

נגדיר $f(n) \in \Omega(g(n))$ אם ורק אם קיימים קבועים $c > 0$ ו- $n_0 \geq 0$ כך שלכל $n \geq n_0$ מתקיים:

$$f(n) \geq c \cdot g(n)$$

✓ הגדרה: חסם תחתון אסימפטוטי קטן (ω)

נגדיר $f(n) \in \omega(g(n))$ אם ורק אם לכל קבוע $c > 0$ קיים $n_0 \geq 0$ כך שלכל $n \geq n_0$ מתקיים $f(n) > c \cdot g(n)$. לחילופין, באמצעות גבולות:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

✓ הגדרה: חסם הדוק אסימפטוטי (Θ)

נגדיר $f(n) \in \Theta(g(n))$ אם ורק אם קיימים קבועים $c_1, c_2 > 0$ ו- $n_0 \geq 0$ כך שלכל $n \geq n_0$ מתקיים:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

פורמלית: $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$. לחילופין, באמצעות גבולות:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = x \in \mathbb{R}^+$$

(כאשר x הוא קבוע חיובי סופי שאינו אפס).

אינטואיטיבית, ניתן להקביל את הסימונים הללו ליחסי סדר מתמטיים:

- הסימון O מקביל ל- $\leq$.
- הסימון o מקביל ל- $<$.
- הסימון Ω מקביל ל- $\geq$.
- הסימון ω מקביל ל- $>$.
- הסימון Θ מקביל ל- $=$.

⚠ הערה: מגבלת ההקבלה ליחסי סדר

ההקבלה ליחסי סדר אינה מוחלטת. במספרים אריתמטיים מתקיים חוק הטריכוטומיה (לכל x, y מתקיים $x < y$, $x = y$ או $x > y$). לעומת זאת, קיימות פונקציות $f(n), g(n)$ שאינן בנות השוואה אסימפטוטית כלל (לדוגמה, פונקציות אוסצילטוריות כמו $f(n) = n^2$ ו- $g(n) = n^{2+\sin n}$).

תכונות של חסמים אסימפטוטיים

1. **רפלקסיביות:** היחסים O, Ω, Θ הם רפלקסיביים. כלומר, $f(n) \in O(f(n))$, $f(n) \in \Theta(f(n))$ ו- $f(n) \in \Omega(f(n))$.
2. **סימטריות:** היחס Θ הוא סימטרי. אם $f(n) \in \Theta(g(n))$ אז $g(n) \in \Theta(f(n))$. היחסים O, Ω אינם סימטריים, אלא מקיימים דואליות: $f(n) \in O(g(n)) \iff g(n) \in \Omega(f(n))$.
3. **טרנזיטיביות:** כל החסמים האסימפטוטיים ($O, o, \Omega, \omega, \Theta$) הם טרנזיטיביים. למשל, אם $f(n) \in O(g(n))$ ו- $g(n) \in O(h(n))$ אז $f(n) \in O(h(n))$.
4. **חיבור פונקציות:** סכום של פונקציות חסום על ידי המקסימום מביניהן:

$$f_1(n) + f_2(n) \in \Theta(\max\{f_1(n), f_2(n)\})$$
תכונה זו נכונה גם עבור חסמי O ו- Ω . לדוגמה: $n^2 + n \log n \in \Theta(n^2)$.
5. **כפל פונקציות:** מכפלת פונקציות שומרת על יחס הכפל האסימפטוטי:

$$f_1(n) \cdot f_2(n) \in \Theta(f_1(n) \cdot g(n))$$
6. **חזקות:** אם $f(n) \in o(g(n))$ אז $a^{f(n)} \in o(a^{g(n)})$ לכל $a > 1$.
7. **לוגריתמים:** אם $f(n) \in \Theta(g(n))$ ופונקציות אלו שואפות לאינסוף, אז $\log(f(n)) \in \Theta(\log(g(n)))$.

טורים וסדרות ידועים

1. הטור ההרמוני:

$$\sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \dots + \frac{1}{n} = \ln n + O(1) \in \Theta(\log n)$$

2. טור גיאומטרי (הנדסי):

עבור מנה $x \neq 1$:

סיכום מקיף במבני נתונים ואלגוריתמים

$$\sum_{i=0}^n x^i = \frac{x^{n+1} - 1}{x - 1}$$

• אם $|x| < 1$, הטור מתכנס לקבוע מלעיל כאשר $n \rightarrow \infty$:

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x} \in \Theta(1)$$

• אם $x > 1$, הטור נשלט על ידי האיבר האחרון שלו:

$$\sum_{i=0}^n x^i \in \Theta(x^n)$$

3. טור מעריכי פולינומי:

$$\sum_{i=1}^n i^k = 1^k + 2^k + \dots + n^k \in \Theta(n^{k+1})$$

- מקרה פרטי $k = 1$ (סדרה חשבונית): $\sum_{i=1}^n i = \frac{n(n+1)}{2} \in \Theta(n^2)$
- מקרה פרטי $k = 2$ (סדרת ריבועים): $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} \in \Theta(n^3)$
- מקרה פרטי $k = 3$ (סדרת מעוקבים): $\sum_{i=1}^n i^3 = \frac{n^2(n+1)^2}{4} \in \Theta(n^4)$

4. טורים של לוגריתמים:

- על פי קירוב סטירלינג למציאת חסם עבור עצרת: $\log(n!) = \Theta(n \log n)$
- סכום של לוגים:

$$\sum_{i=1}^n \log(i) = \log(1 \cdot 2 \cdot \dots \cdot n) = \log(n!) \in \Theta(n \log n)$$

- סכום לוגריתמים עם בסיס משתנה (שיטת הפירוק לחצאים):

$$\sum_{i=2}^n \log_i n = \Theta(n)$$

הוכחת חסם זה מבוססת על חלוקת הטור לשני חצאים (מ-2 עד $\sqrt{n}$ ומ- $\sqrt{n} + 1$ עד n) ושימוש בחסמים אריתמטיים עליונים ותחתונים.

שלבי עבודה מעשיים לניתוח סיבוכיות

1. **זיהוי הקלט:** הגדרת גודל הקלט n באופן מדויק. במערכים דו-ממדיים בגודל $n \times n$, גודל הקלט הכולל הוא $N = n^2$. לכן, אלגוריתם הרץ בזמן $O(n^2)$ הוא למעשה ליניארי ביחס לגודל הקלט הכולל N , כלומר $O(N)$.
2. **ניתוח קריאות לפונקציות מובנות:** יש לשים לב לסיבוכיות של פונקציות ספריה כגון `strlen` במחרוזות, הדורשת סריקה ליניארית של כל המחרוזות ולכן עלותה $O(n)$.

3. ניתוח לולאות מקוננות:

- **לולאות בלתי תלויות:** אם מספר האיטרציות בלולאה הפנימית אינו תלוי באינדקס הלולאה החיצונית, נחשב את סיבוכיות כל אחת מהן בנפרד ונכפול את התוצאות.
- **לולאות תלויות:** אם מספר האיטרציות בלולאה הפנימית תלוי באינדקס הלולאה החיצונית, נבטא את מספר הפעולות הכולל כסכום (טור מתמטי) ונפתור אותו אסימפטוטית.

⚠ הערה: משתנה חסום כקבוע

אם במהלך ניתוח לולאה מופיע משתנה d המקבל ערכים שונים בכל איטרציה, אך ערכו חסום תמיד בין שני קבועים (לדוגמה: $5 \leq d \leq 10$), ניתן להתייחס אליו בניתוח האסימפטוטי כאל קבוע $O(1)$.

משפט הסנדוויץ' לחישוב חסמים אסימפטוטיים הדוקים

משפט הסנדוויץ' מנוסח באופן אסימפטוטי כדי להוכיח שפונקציה $f(n)$ שייכת ל- $\Theta(g(n))$.

✓ משפט: משפט הסנדוויץ' האסימפטוטי

תהייה a_n, b_n, c_n שלוש סדרות חיוביות. אם לכל $n \geq n_0$ מתקיים $a_n \leq b_n \leq c_n$, ואם מתקיים ש-
 $a_n \in \Theta(g(n))$ וגם $c_n \in \Theta(g(n))$, אזי בהכרח מתקיים:

$$b_n \in \Theta(g(n))$$

פעמים רבות קל יותר למצוא חסם עליון $O(g(n))$ וחסם תחתון $\Omega(h(n))$ עבור פונקציות עזר פשוטות יותר החוסמות את פונקציית המטרה, וכאשר $g(n) = h(n)$, לקבל חסם הדוק Θ באופן עקיף.

2. משוואות רקורסיה ואלגוריתמים רקורסיביים

באלגוריתמים רקורסיביים נרצה לבטא את מספר הפעולות הכולל $T(n)$ כפונקציה המורכבת מהעלות של הקריאות החוזרות בתוספת העלות הכללית שבוצעה בקריאה הנוכחית ללא הרקורסיה (המסומנת כ- $f(n)$).

שיטת העץ (Iteration Tree Method)

בשיטה זו מייצגים את הקריאות הרקורסיביות כעץ קריאות, שבו כל קודקוד מייצג קריאה בודדת ומכיל את העלות הישירה שבוצעה בה $(f(n))$.

כדי לפתור את משוואת הרקורסיה באמצעות שיטה זו, נחשב שני פרמטרים מרכזיים:

1. **עומק העץ (גובה הרקורסיה):** מספר הצעדים הנדרש כדי להגיע ממפתן הקלט n למקרה הבסיס (לרוב $T(1)$ או $T(0)$).

2. **סכום העבודה בכל רמה:** סכימת כל הערכים של הקודקודים ברמה ה- i של העץ. בסיום, נסכום את העבודה של כל רמות העץ כדי לקבל את החסם הכולל.

⚠ הערה: התכנסות סדרות הנדסיות בעץ

כאשר סכומי הרמות בעץ מהווים סדרה הנדסית יורדת בעלת מנה קבועה הקטנה מ-1 (לדוגמה: בכל רמה מבצעים עבודה של $(\frac{5}{16} \cdot n)^i$), סכום הטור כולו נשלט על ידי עבודת שורש העץ, והסיבוכיות הכוללת היא ליניארית $O(n)$ ללא תלות ישירה בגובה העץ. לעומת זאת, אם סכום העבודה בכל רמה הוא קבוע (לדוגמה: n), העבודה הכוללת תהיה מכפלת גובה העץ בעבודה הקבועה לרמה, כלומר $O(n \log n)$.

שיטת ההצבה (Substitution Method)

שיטה זו מורכבת משני שלבים:

1. **ניחוש החסם:** ביצוע הערכה לגבי סדר הגודל של הפתרון (לרוב מבוסס על אינטואיציה משיטת העץ).
2. **הוכחה פורמלית באינדוקציה:** הוכחה שההערכה נכונה עבור כל n על ידי הנחת נכונות עבור $n' < n$ והראית קיומם של קבועים מתאימים.

דוגמה להוכחה בשיטת ההצבה

נוכיח כי הפתרון למשוואה $T(n) = 2T(\lfloor \frac{n}{2} \rfloor) + n$ הוא $T(n) \in O(n \log n)$.

הוכחה:

עלינו להראות שקיים קבוע $c > 0$ כך ש- $T(n) \leq cn \log n$ לכל $n \geq n_0$.
נניח באינדוקציה כי החסם מתקיים לכל ערך קטן מ- n , ובפרט עבור $\lfloor \frac{n}{2} \rfloor$.
נציב במשוואה המקורית:

$$T(n) = 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n \leq 2\left(c \left\lfloor \frac{n}{2} \right\rfloor \log \left\lfloor \frac{n}{2} \right\rfloor\right) + n$$

נשתמש בתכונה $\lfloor \frac{n}{2} \rfloor \leq \frac{n}{2}$:

$$T(n) \leq 2\left(c \frac{n}{2} \log \left(\frac{n}{2}\right)\right) + n = cn \log \left(\frac{n}{2}\right) + n$$

$$T(n) \leq cn(\log n - \log 2) + n = cn \log n - cn \log 2 + n$$

נניח כי בסיס הלוגריתם הוא 2, ולכן $\log_2 2 = 1$:

$$T(n) \leq cn \log n - cn + n$$

עלינו לדרוש שביטוי זה יהיה קטן או שווה ל- $cn \log n$:

$$cn \log n - cn + n \leq cn \log n \iff n \leq cn \iff c \geq 1$$

האי-שוויון מתקיים לכל $c \geq 1$. כעת נותר לבחור את בסיס האינדוקציה n_0 המקיים את החסם בפועל. נשים לב שעבור $n = 1$ מתקיים $c \cdot 1 \cdot \log 1 = 0$, מה שלא יאפשר חסימה של $T(1) > 0$. לכן, נבחר כבסיס האינדוקציה את המקרים $n = 2$ ו- $n = 3$ ונמצא קבוע c גדול מספיק שיענה על תנאי ההתחלה שלהם.

שיטת המאסטר (Master Theorem)

משפט המאסטר מספק פתרון מיידי למשוואות רקורסיה מהצורה הבאה:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

כאשר $a \geq 1$ הוא מספר תתי-הבעיות בכל שלב, $b > 1$ הוא פקטור צמצום הקלט, ו- $f(n)$ היא פונקציית העבודה הישירה בכל קריאה.

נשווה אסימפטוטית בין פונקציית העבודה הישירה $f(n)$ לבין האיבר הדומיננטי של העץ $n^{\log_b a}$:

משפט: משפט המאסטר

1. **מקרה 1:** אם $f(n) \in O(n^{\log_b a - \epsilon})$, עבור קבוע $\epsilon > 0$, אזי הפתרון נשלט על ידי עלי הבסיס של העץ:

$$T(n) \in \Theta(n^{\log_b a})$$

2. **מקרה 2:** אם $f(n) \in \Theta(n^{\log_b a})$, אזי העבודה מתחלקת באופן שווה בין כל רמות העץ, והפתרון הוא:

$$T(n) \in \Theta(n^{\log_b a} \log n)$$

3. **מקרה 3:** אם $f(n) \in \Omega(n^{\log_b a + \epsilon})$, עבור קבוע $\epsilon > 0$, ואם מתקיים תנאי הרגולריות $af(\frac{n}{b}) \leq cf(n)$ עבור קבוע $c < 1$ לכל n גדול מספיק, אזי הפתרון נשלט על ידי עבודת השורש:

$$T(n) \in \Theta(f(n))$$

שיטת החלפת משתנים (Change of Variables)

במשוואות רקורסיה שאינן ליניאריות בקלט (כמו קיום שורש בקלט), ניתן לבצע החלפת משתנים כדי להמירן לצורה מוכרת הניתנת לפתרון בשיטות סטנדרטיות.

דוגמה לפתרון באמצעות החלפת משתנים

נתונה הרקורסיה: $T(n) = T(\sqrt{n}) + 1$.

נבצע שינוי משתנה. נסמן $n = 2^m$, כלומר $m = \log_2 n$. נציב במשוואה:

$$T(2^m) = T(2^{\frac{m}{2}}) + 1$$

נגדיר פונקציה חדשה $S(m) = T(2^m)$:

$$S(m) = S\left(\frac{m}{2}\right) + 1$$

על פי משפט המאסטר (מקרה 2, שכן $a=1, b=2, f(m)=1 \implies m^{\log_2 1} = m^0 = 1$), נקבל:

$$S(m) \in \Theta(\log m)$$

נחזיר את המשתנה המקורי על ידי הצבה של $m = \log_2 n$:

$$T(n) = S(\log n) \in \Theta(\log(\log n))$$

3. ניתוח לשיעורין (Amortized Analysis)

ניתוח לשיעורין

ניתוח לשיעורין הוא שיטה להערכת זמן הריצה של סדרת פעולות במבנה נתונים, במטרה לקבל את העלות הממוצעת של פעולה בודדת במקרה הגרוע ביותר. בניגוד לניתוח ממוצע רגיל, כאן אין הנחות הסתברותיות לגבי התפלגות הקלטים, אלא מובטחת עלות ממוצעת נמוכה לאורך סדרה ארוכה של פעולות במקרה הגרוע.

קיימות שלוש שיטות מרכזיות לביצוע ניתוח לשיעורין:

1. שיטת הצבירה (Aggregate Method)

בשיטה זו מחשבים באופן ישיר את החסם העליון של העלות הכוללת עבור סדרה של n פעולות במקרה הגרוע ביותר, המסומנת כ- $T(n)$. לאחר מכן, מגדירים את העלות לשיעורין (המשוערכת) של פעולה בודדת כחלוקת העלות הכוללת במספר הפעולות:

$$\hat{c} = \frac{T(n)}{n}$$

2. שיטת החיובים / הבנקאי (Accounting Method)

בשיטה זו אנו מייחסים לכל פעולה עלות משוערכת $\hat{c}_i$ השונה מהעלות האמיתית שלה c_i :

- לפעולות "זולות" (שעלותן האמיתית נמוכה מהעלות המשוערכת שקבענו) אנו גובים מחיר משוערך גבוה יותר, ואת העודף ($\hat{c}_i - c_i > 0$) אנו מפקידים כ"אשראי" (Credit) בבנק של מבנה הנתונים.
- לפעולות "יקרות" (שעלותן האמיתית גבוהה מהעלות המשוערכת שקבענו) אנו משלמים מחיר משוערך נמוך, ומכסים את ההפרש האמיתי על ידי משיכת האשראי שנצבר בבנק מהפעולות הזולות הקודמות.

⚠ תנאי הכרחי בשיטת החיובים

כדי שהניתוח יהיה תקף, מאזן האשראי המצטבר בבנק חייב להישאר תמיד אי-שלילי בכל שלב במהלך הרצת סדרת הפעולות. כלומר, לכל $k \geq 1$:

$$\sum_{i=1}^k \hat{c}_i \geq \sum_{i=1}^k c_i$$

תנאי זה מבטיח שהעלות המשוערכת הכוללת מהווה תמיד חסם מלעיל לעלות האמיתית הכוללת של האלגוריתם.

3. שיטת הפוטנציאל (Potential Method)

בשיטה זו אנו מגדירים פונקציית פוטנציאל Φ הממפה את המצב של מבנה הנתונים D למספר ממשי חיובי: $\Phi(D) \geq 0$. הפוטנציאל מייצג את כמות "האנרגיה" או האשראי האגור במבנה הנתונים שיכול לשמש למימון פעולות יקרות בעתיד.

יהי D_0 המצב ההתחלתי של מבנה הנתונים, ויהי D_i המצב לאחר הפעולה ה- i . נגדיר את העלות המשוערכת $\hat{c}_i$ של הפעולה ה- i באופן הבא:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

כאשר c_i היא העלות האמיתית של הפעולה, וההפרש $\Delta\Phi = \Phi(D_i) - \Phi(D_{i-1})$ הוא השינוי בפוטנציאל בעקבות ביצועה.

נסכום את העלויות המשוערכות לאורך סדרה של n פעולות (טור טלסקופי):

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$$

אם נבטיח כי לכל i מתקיים $\Phi(D_i) \geq \Phi(D_0)$ (ובפרט אם נבחר $\Phi(D_0) = 0$), אזי מתקבל:

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

מכאן שחסימת העלות המשוערכת $\hat{c}_i$ מלמעלה חוסמת ישירות גם את העלות הממוצעת האמיתית לשיעורין.

דוגמאות ויישומים מעשיים

א. מחסנית משודרגת (Multi-Pop Stack)

נוסף לפעולות הרגילות Push ו-Pop בעלות $O(1)$, המחסנית תומכת בפעולה $\text{Multi-Pop}(k)$ המסירה את k האיברים העליונים מהמחסנית. עלותה האמיתית של פעולה זו היא $\min(k, s)$ כאשר s הוא מספר האיברים הנוכחי במחסנית. במקרה הגרוע ביותר, עלות פעולת Multi-Pop יחידה היא $O(n)$.

ננתח סדרת פעולות של n איברים המתחילה ממחסנית ריקה:

ניתוח בשיטת הצבירה

נשים לב שכל איבר יכול להיגרע מהמחסנית לכל היותר פעם אחת עבור כל פעם שהוא הוכנס אליה. מכיוון שמספר פעולות ה-Push בסדרה של n פעולות חסום על ידי n , מספר פעולות ה-Pop וה- Multi-Pop המצטברות לאורך כל הסדרה חסום בהכרח על ידי n .

לפיכך העלות הכוללת של n פעולות היא לכל היותר $T(n) \leq 2n$. העלות לשיעורין של פעולה בודדת היא:

$$\hat{c} = \frac{T(n)}{n} \leq \frac{2n}{n} = 2 \in O(1)$$

ניתוח בשיטת החיובים

נגדיר את העלויות המשוערכות הבאות:

- עבור Push : נגבה עלות משוערכת של $\hat{c} = 2$. שקל אחד מתוכם משמש למימון פעולת ההכנסה האמיתית, ושקל אחד מופקד כאשראי על גבי האיבר שהוכנס.
- עבור Pop : נגבה עלות משוערכת של $\hat{c} = 0$. הפעולה ממומנת לחלוטין על ידי משיכת השקל שהופקד על גבי האיבר בעת הכנסתו.
- עבור $\text{Multi-Pop}(k)$: נגבה עלות משוערכת של $\hat{c} = 0$. כל אחת מ- k המחיקות בפועל ממומנת על ידי השקלים המונחים על גבי האיברים שמוסרים מהמחסנית.

מאחר שלעולם לא ניתן למחוק איבר שאינו קיים במחסנית, בבנק תמיד יישאר מאזן אי-שלילי. העלות המשוערכת של כל פעולה היא לכל היותר 2, ולכן העלות לשיעורין היא $O(1)$.

ניתוח בשיטת הפוטנציאל

נגדיר את פונקציית הפוטנציאל $\Phi(S)$ כאל מספר האיברים הנוכחי במחסנית S . עבור מחסנית ריקה, מתקיים $\Phi(D_0) = 0$. ברור כי לכל מצב D_i מתקיים $\Phi(D_i) \geq 0$. ננתח את העלויות המשוערכות של הפעולות:

• פעולת Push :

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + (s + 1) - s = 2$$

• פעולת Pop :

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + (s - 1) - s = 0$$

• פעולת Multi-Pop(k) : נסמן $k' = \min(k, s)$.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = k' + (s - k') - s = 0$$

בכל המקרים העלות המשוערכת היא $O(1)$, ומכאן שהעלות לשיעורין היא $O(1)$.

ב. מונה בינארי (Binary Counter)

מונה בינארי בגודל k סיביות מיוצג כמערך $A[0 \dots k-1]$. נרצה לממש פעולת Increment המגדילה את המונה ב-1. במקרה הגרוע, כל k הביטים יכולים להתהפך (לדוגמה מעבר מ-0111 ל-1000), ולכן עלות הפעולה במקרה הגרוע היא $O(k)$.

ניתוח בשיטת הצבירה

נבחין בתדירות ההיפוך של כל ביט בנפרד במהלך סדרה של n פעולות Increment :

- הביט $A[0]$ מתהפך בכל פעולת קידום (סך הכל n פעמים).
- הביט $A[1]$ מתהפך בכל פעולה שנייה (סך הכל $\lfloor \frac{n}{2} \rfloor$ פעמים).
- הביט $A[i]$ מתהפך בכל פעולה של 2^i (סך הכל $\lfloor \frac{n}{2^i} \rfloor$ פעמים).

העלות הכוללת של n פעולות היא סכום ההיפוכים המצטבר:

$$T(n) = \sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

העלות לשיעורין של פעולת קידום בודדת היא:

$$\hat{c} = \frac{T(n)}{n} < \frac{2n}{n} = 2 \in O(1)$$

ניתוח בשיטת החיובים

בכל פעולת Increment אנו הופכים רצף של ביטים שערכם 1 ל-0, וביט יחיד שערכו 0 ל-1. נגדיר עלות משוערכת של $\hat{c} = 2$:

- שקל אחד משמש למימון שינוי ה-0 ל-1 בפועל.
- שקל אחד מופקד כאשראי על גבי הביט שהפך ל-1, לשימוש עתידי כאשר הוא יצטרך להתהפך בחזרה ל-0.
- מחירי הפיכת ה-1 ל-0 ממומנים באופן מלא על ידי משיכת השקל המונח עליהם.

מאחר שכל ביט שערכו 1 מחזיק בהכרח שקל אשראי שהופקד עליו בעת הפיכתו ל-1, לא נגיע לעולם לגרעון. העלות לשיעורין היא $O(1)$.

ניתוח בשיטת הפוטנציאל

נגדיר את פונקציית הפוטנציאל $\Phi(D_i)$ כמספר הביטים שערכם 1 במונה במצב D_i . עבור מונה מאותחל ב-0, מתקיים $\Phi(D_0) = 0$. ברור כי לכל מצב מתקיים $\Phi(D_i) \geq 0$. בפעולה ה- i , נניח כי הפכנו רצף של t ביטים שערכם 1 ל-0, וביט יחיד מ-0 ל-1. העלות האמיתית של הפעולה היא $c_i = t + 1$. מספר הביטים שערכם 1 במצב החדש הוא:

$$\Phi(D_i) = \Phi(D_{i-1}) - t + 1$$

העלות המשוערכת של הפעולה היא:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = (t + 1) + (\Phi(D_{i-1}) - t + 1) - \Phi(D_{i-1}) = 2 \in O(1)$$

ג. מערך דינמי (Dynamic Array)

מערך דינמי מתחיל מגודל קבוע קטן (לרוב תא בודד), וכאשר הוא מתמלא, מוקצה מערך חדש כפול בגודלו, וכל האיברים מועתקים אליו. פעולת הכנסה רגילה ללא הכפלה עולה $O(1)$, אך פעולת הכנסה הגוררת הכפלה דורשת הקצאה והעתקה של i איברים, ולכן עלותה האמיתית היא $c_i = i + 1$. במקרה הגרוע, פעולה בודדת עולה $O(n)$.

ניתוח בשיטת הצבירה

בסדרה של n הכנסות, הכפלת מערך מתרחשת רק כאשר גודל המערך הנוכחי הוא חזקה של 2. נסכום את העלויות של כל הפעולות:

$$T(n) = \sum_{i=1}^n c_i = n + \sum_{j=0}^{\lfloor \log n \rfloor} 2^j < n + 2^{\lfloor \log n \rfloor + 1} \leq n + 2n = 3n$$

העלות לשיעורין של הכנסה בודדת היא:

$$\hat{c} = \frac{T(n)}{n} < \frac{3n}{n} = 3 \in O(1)$$

ניתוח בשיטת החיובים

נגדיר עלות משוערכת של $\hat{c} = 3$ לכל פעולת הכנסה:

- שקל אחד משמש להכנסת האיבר עצמו למערך.
- שקל אחד מופקד כאשר על גבי האיבר שהוכנס כעת, כדי לממן את העתקתו העתידית בעת הכפלת המערך הבאה.
- שקל אחד מופקד כאשר איבר ישן שכבר נמצא במחצית הראשונה של המערך ושאינו לו אשראי, כדי לממן את העתקתו העתידית (שכן בכל הכפלה, אנו מכפילים את כמות האיברים ולכן כל איבר חדש שנכנס צריך לממן את העתקתו שלו ואת העתקתו של איבר ותיק אחד).

בעת הגעה לשלב ההכפלה, לכל האיברים במערך יש שקל אחד המונח עליהם, המממן באופן מדויק את עלות ההעתקה האמיתית. לכן מאזן הבנק נשאר חיובי, והעלות לשיעורין היא $O(1)$.

ניתוח בשיטת הפוטנציאל

יהי s מספר האיברים הנוכחי במערך, ויהי w גודל המערך המוקצה הכולל (Capacity). נגדיר את פונקציית הפוטנציאל באופן הבא:

$$\Phi(D) = 2s - w$$

- מיד לאחר הכפלת מערך מתקיים $w = 2s$, ולכן $\Phi(D) = 0$.
- כאשר המערך מלא לחלוטין מתקיים $w = s$, ולכן $\Phi(D) = s$, מה שמבטיח פוטנציאל גבוה המספיק לממן את ההעתקה הבאה.

- ברור כי לכל מצב מתקיים $\Phi(D_i) \geq 0$ (שכן תמיד המערך מלא לפחות בחצי מגודלו המוקצה).

ננתח את העלות המשוערכת של פעולת הכנסה:

1. **מקרה א' (ללא הכפלה):** גודל המערך w אינו משתנה. העלות האמיתית היא $c_i = 1$.

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + (2s_i - w) - (2(s_i - 1) - w) = 3$$

2. **מקרה ב' (עם הכפלה):** מתבצעת הכפלה מ- w ל- $2w$. העלות האמיתית היא $c_i = s_i + 1$ (העתקת $s_i - 1$ איברים קודמים והכנסת האיבר החדש).

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = (s_i + 1) + (2s_i - 2w) - (2(s_i - 1) - w)$$

נשים לב שלפני ההכפלה המערך היה מלא, ולכן התקיים $s_i - 1 = w$. נציב זאת:

$$\hat{c}_i = (s_i + 1) + 2s_i - 2(s_i - 1) - (2(s_i - 1) - (s_i - 1)) = s_i + 1 + 2 - (s_i - 1) = 3 \in O(1)$$

דה-אמורטיזציה (Deamortization)

דה-אמורטיזציה היא טכניקה מתקדמת במבני נתונים שמטרתה להמיר מבנה נתונים בעל סיבוכיות טובה לשיעורין (Amortized) למבנה נתונים בעל סיבוכיות מובטחת במקרה הגרוע ביותר הישיר (Worst Case). הרעיון הכללי הוא לבצע פעולות יקרות (כגון הכפלת מערך או בנייה מחדש) באופן הדרגתי ואינקרמנטלי לאורך מספר רב של פעולות זולות, במקום לבצע את כל העבודה היקרה בבת אחת. לדוגמה, במערך דינמי, בכל הכנסה אנו מקצים מראש את המערך הכפול ומעתיקים אליו חלק קטן מהאיברים בכל שלב ברקע, כך שברגע שהמערך הנוכחי מתמלא, המערך החדש כבר מוכן לחלוטין לפעילות ללא השהייה חריגה.

4. מבני נתונים בסיסיים ורשימות מקושרות

טיפוס נתונים אבסטרקטי (ADT) מול מימוש פיזי

- **ADT (Abstract Data Type):** הגדרה מתמטית תיאורטית של מבנה הנתונים, המפרטת את סוגי המידע, הנשמרים בו ואת סט הפעולות הנתמכות וממשקן, ללא התייחסות לדרך המימוש הפיזית בזיכרון (לדוגמה: מילון, תור עדיפויות).
- **מימוש פיזי (Data Structure):** המימוש הקונקרטי בפועל בזיכרון המחשב (לדוגמה: מערך, רשימה מקושרת, עץ חיפוש בינארי).

אלגוריתמי חיפוש במערך ממזין

נניח כי נתון מערך ממין בסדר עולה $A[0 \dots n-1]$.

1. חיפוש בינארי (Binary Search)

האלגוריתם מחלק את טווח החיפוש לשני חצאים בכל שלב על ידי השוואה לאיבר האמצעי. סיבוכיות זמן: $O(\log n)$. סיבוכיות מקום: $O(1)$ במימוש איטרטיבי, או $O(\log n)$ במימוש רקורסיבי בשל מחסנית הקריאות.

2. חיפוש מקביל (Parallel Search)

בהינתן מספר מעבדים או אפשרות להרצה במקביל, נריץ במקביל חיפוש ליניארי מתחילת המערך וחיפוש בינארי. אם האיבר המבוקש נמצא באינדקס קטן k , החיפוש הליניארי יסיים מהר, ואם הוא נמצא באינדקס גדול, החיפוש הבינארי יסיים מהר. סיבוכיות זמן כוללת: $O(\min\{k, \log n\})$.

3. חיפוש אקספוננציאלי (Exponential Search)

חיפוש זה מתאים למצבים בהם גודל המערך אינו ידוע מראש או שהוא אינסופי, ונרצה למצוא איבר הנמצא באינדקס k כלשהו. האלגוריתם מוצא את התא הראשון שמיקומו הוא חזקה של 2 ($2^0, 2^1, 2^2 \dots$) שבו הערך גדול או שווה לערך המבוקש x . נניח כי עצרנו בחזקה 2^i . מתקיים:

$$2^{i-1} < k \leq 2^i \implies 2^i < 2k$$

מכאן שגובה החזקה הוא $i \in O(\log k)$. כעת נבצע חיפוש בינארי בטווח שבין 2^{i-1} ל- 2^i , שגודלו הוא לכל היותר $2^i \in O(k)$. החיפוש הבינארי בטווח זה ידרוש זמן של $O(\log k)$. סיבוכיות זמן כוללת: $O(\log k)$.

בעיות סכום: Sum-2 ו-Sum-3

בעיית Sum-2

נתונה קבוצת מספרים S וערך מטרה y . נרצה לקבוע האם קיימים $a, b \in S$ כך ש- $a + b = y$.

השוואת גישות מימוש ל-Sum-2

1. מערך לא ממין (גישה נאיבית): מעבר על כל זוג איברים אפשרי במערך באמצעות לולאות כפולות.
 - זמן ריצה: $O(n^2)$. מקום: $O(1)$.
2. חיפוש בינארי במערך ממין: נמין תחילה את המערך ב- $O(n \log n)$. לכל איבר $x \in S$, נחפש את האיבר המשלים $y - x$ באמצעות חיפוש בינארי ב- $O(\log n)$.
 - זמן ריצה: $O(n \log n)$. מקום: $O(1)$.
3. שיטת שני המצביעים (מערך ממין): נמין את המערך. נציב מצביע שמאלי ℓ בתחילת המערך ומצביע ימני h בסופו. בכל שלב נבחן את הסכום $A[\ell] + A[h]$:
 - אם הסכום שווה ל- y , מצאנו את הזוג וסיימנו.
 - אם הסכום קטן מ- y , עלינו להגדיל את הסכום ולכן נקדם את המצביע השמאלי $\ell \leftarrow \ell + 1$.

- אם הסכום גדול מ- y , עלינו להקטין את הסכום ולכן נסוג עם המצביע הימני $h \leftarrow h - 1$.
- זמן ריצה: $O(n \log n)$ עבור המיון, וסריקה ליניארית של $O(n)$. סך הכל $O(n \log n)$. מקום: $O(1)$.
- 4. **שימוש בטבלת גיבוב (Hash Table):** נכניס את כל האיברים לטבלת גיבוב ב- $O(n)$. לכל איבר x , נבצע שאלתת חיפוש בטבלה עבור האיבר המשלים $y - x$ ב- $O(1)$ בממוצע.
- זמן ריצה: $O(n)$ בממוצע. מקום: $O(n)$.

בעיית Sum-3

לקבוע האם קיימים שלושה איברים $a, b, c \in S$ כך ש- $a + b + c = y$.

גישת מימוש אופטימלית ל-Sum-3

נמיון את המערך ב- $O(n \log n)$. נריץ לולאה חיצונית מ- $i = 0$ עד $n - 3$. בכל איטרציה נבחר את האיבר $A[i]$ להיות האיבר הראשון בשלישייה, ונפתור את בעיית ה-Sum-2 עבור תת-המערך שמימין לו $A[i + 1 \dots n - 1]$ עם ערך מטרה מבוקש של $y - A[i]$. נשתמש בשיטת שני המצביעים לפתרון ה-Sum-2 בטווח זה בזמן $O(n)$. סיבוכיות זמן כוללת: $O(n^2)$. סיבוכיות מקום: $O(1)$ (או בהתאם למיון שנבחר).

רשימות מקושרות

רשימה מקושרת היא מבנה נתונים דינמי שבו האיברים מפוזרים בזיכרון (בניגוד למערך שבו האיברים רציפים בזיכרון). כל איבר (Node) מכיל שדה מידע (Data) ומצביע (Pointer) לאיבר הבא ברשימה.

⚠ הערה: אחסון מצביעים למידע ברשימה

מומלץ ששדה המידע באיבר של הרשימה המקושרת יכיל מצביע למידע עצמו ולא את ערך המידע ישירות. בדרך זו, אם נרצה למיין את הרשימה או לשנות את סדר האיברים, נצטרך רק לשנות את המצביעים הפנימיים למידע מבלי להעתיק או להזיז את המידע הפיזי עצמו בזיכרון, פעולה החוסכת זמן ריצה ומשאבים.

סוגי רשימות מקושרות

- **חד-כיוונית (Singly Linked List):** כל איבר מצביע רק על האיבר הבא אחריו.
- **דו-כיוונית (Doubly Linked List):** כל איבר מכיל מצביע לאיבר הבא אחריו ומצביע לאיבר הקודם לו. מאפשרת תנועה דו-כיוונית ונוחות במחיקת איברים ללא צורך במעקב אחר האיבר הקודם, במחיר של זיכרון נוסף עבור מצביעים.
- **מעגלית (Circular Linked List):** האיבר האחרון ברשימה אינו מצביע ל-Null, אלא מצביע בחזרה לאיבר הראשון. ברשימה כזו אין הגדרה מוחלטת ל"ראש" (Head) או ל"זנב" (Tail), וניתן להגיע מכל איבר לכל איבר בצורה רציפה.

🔗 זקיף (Sentinel)

זקיף הוא איבר מיוחד (ללא מידע) הממוקם כראש הרשימה (לעיתים גם בסופה). שימוש בזקיף משמש כ"עוגן" קבוע בזיכרון, המונע את הצורך בטיפול במקרי קצה מיוחדים (כגון מחיקה או הכנסה של האיבר הראשון המאלצת עדכון של מצביע ה-Head), ובכך מפשט את מימוש הקוד ומונע שגיאות של גישה למצביעי Null.

אלגוריתמים מתקדמים ברשימות מקושרות

1. מציאת נקודת מפגש (התמזגות) של שתי רשימות מקושרות

נתונות שתי רשימות L_1, L_2 המתמזגות לנקודת מפגש משותפת x (מבנה בצורת האות Y). נסמן את אורכי הרשימות כ- $k+n$ ו- $l+n$, כאשר n הוא אורך הקטע המשותף, ו- k, l הם אורכי הקטעים הייחודיים של כל רשימה לפני נקודת המפגש.

גישה א': גישור על הפרש $O(k+l+n)$ זמן, $O(1)$ מקום)

נמדוד את אורכי שתי הרשימות על ידי סריקתן עד הסוף. נחשב את הפרש האורכים

$$d = |(k+n) - (l+n)| = |k-l|$$

נציב שני מצביעים בראש הרשימות. את המצביע של הרשימה הארוכה יותר נקדם d צעדים קדימה. כעת, שני המצביעים נמצאים במרחק שווה מנקודת המפגש x .

נקדם את שני המצביעים במקביל צעד אחד בכל פעם, עד אשר הם יצביעו על אותו איבר משותף. איבר זה הוא בהכרח נקודת המפגש x .

גישה ב': חיפוש אקספוננציאלי $O(\max\{k, l\})$ זמן, $O(1)$ מקום)

נניח ללא הגבלת הכלליות כי $k \geq l$.

נריץ במקביל סריקה על הרשימה L_1 בקפיצות שהן חזקות של 2 ($2^0, 2^1, 2^2, \dots$). בכל שלב נבחן האם האיבר הנוכחי מופיע ברשימה L_2 באמצעות בדיקה של 2^i איברים. ברגע שנגיע לחזקה הגדולה מ- k , נצמצם את טווח החיפוש ונבצע את החיפוש הגיאומטרי. פתרון זה מייעל את החיפוש כאשר נקודת המפגש קרובה יחסית לראש הרשימה.

2. זיהוי מעגל ברשימה מקושרת ומציאת נקודת תחילת המעגל

נרצה לקבוע האם רשימה מקושרת מכילה מעגל, ובמידה וכן - למצוא את האיבר הראשון שבו המעגל מתחיל.

משפט: אלגוריתם Floyd למציאת מעגל (Tortoise and Hare)

משפט: אלגוריתם Floyd

- זיהוי המעגל:** נגדיר שני מצביעים המתחילים בראש הרשימה: "צב" (מתקדם צעד אחד בכל פעם) ו"ארנב" (מתקדם שני צעדים בכל פעם). אם קיים מעגל ברשימה, הארנב והצב ייפגשו בשלב כלשהו בתוך המעגל. אם הארנב מגיע ל-Null, אין מעגל ברשימה.
- מציאת ראש המעגל:** לאחר נקודת המפגש בתוך המעגל, נחזיר את הצב לראש הרשימה, ונשאיר את הארנב בנקודת המפגש. כעת, נקדם את שניהם בקצב שווה של צעד אחד בכל פעם. נקודת המפגש החדשה שלהם תהיה בדיוק באיבר שבו המעגל מתחיל.

הוכחה:

נסמן ב- a את המרחק מראש הרשימה לתחילת המעגל, ב- b את המרחק מתחילת המעגל לנקודת המפגש הראשונה, וב- L את אורך המעגל הכולל.

בעת המפגש הראשון, הצב עבר מרחק של $x = a + b + p \cdot L$ (עבור מספר סיבובים p).

הארנב, שנע במהירות כפולה, עבר מרחק של $2x = a + b + q \cdot L$ (עבור מספר סיבובים q).

$$2(a + b + p \cdot L) = a + b + q \cdot L \implies a + b = (q - 2p) \cdot L = k \cdot L$$

מכאן $a = k \cdot L - b$

כלומר, המרחק מראש הרשימה לתחילת המעגל (a) שווה בדיוק למרחק מנקודת המפגש הנוכחית לתחילת המעגל (השלמת סיבובים פחות b). לפיכך, אם נניע מצביע אחד מראש הרשימה ומצביע שני מנקודת המפגש באותו קצב, הם ייפגשו בהכרח לאחר מעבר מרחק של a , שזהו בדיוק ראש המעגל.
סיבוכיות זמן: $O(n)$. סיבוכיות מקום: $O(1)$.

5. מחסניות ותורים

מחסנית (Stack)

מחסנית היא מבנה נתונים מסוג **LIFO (Last In First Out)**. האיבר האחרון שהוכנס למחסנית הוא האיבר הראשון שיצא ממנה.

- פעולות נתמכות:** **Push** (הכנסת איבר לראש המחסנית), **Pop** (הסרת איבר מראש המחסנית), **Top** (הצגת האיבר בראש המחסנית ללא הסרתו), **IsEmpty**, **Size**.
- מימושים פיזיים:**
 - מערך:** מהיר ויעיל בזיכרון, אך מוגבל בגודל מקסימלי קבוע מראש (אלא אם משתמשים במערך דינמי הגורר עלויות הכפלה לשיעורין).
 - רשימה מקושרת:** גודל דינמי בלתי מוגבל, הכנסה והסרה מראש הרשימה (כדי לשמור על סיבוכיות $O(1)$ קבועה). דורשת תקורה נוספת של זיכרון עבור מצביעי ה- **Next**.

תור (Queue)

תור הוא מבנה נתונים מסוג **FIFO (First In First Out)**. האיבר הראשון שהוכנס לתור הוא האיבר הראשון שיצא ממנו.

- פעולות נתמכות:** **Enqueue** (הכנסת איבר לסוף התור), **Dequeue** (הסרת האיבר מראש התור), **Front** (הצגת האיבר בראש התור), **IsEmpty**, **Size**.
- מימושים פיזיים:**
 - רשימה מקושרת:** מיוצגת עם מצביעים הן לתחילת הרשימה (עבור **Dequeue**) והן לסופה (עבור **Enqueue**) לשמירה על יעילות של $O(1)$.
 - מערך מעגלי:** שימוש במערך בגודל קבוע N עם שני אינדקסים: h (ראש התור, מיועד להוצאה) ו- t (זנב התור, מיועד להכנסה). מניעת זליגה מהמערך מתבצעת על ידי חישוב אינדקסים מודולרי:

$$t = (t + 1) \bmod N$$

$$h = (h + 1) \bmod N$$

התור מלא כאשר מתקיים $(t + 1) \bmod N = h$, והוא ריק כאשר מתקיים $t = h$.

מימוש תור באמצעות שתי מחסניות (Two-Stack Queue)

נרצה לממש תור תקין באמצעות שימוש בשתי מחסניות בלבד: מחסנית כניסה S_1 ומחסנית יציאה S_2 .

אלגוריתם המימוש

- פעולת Enqueue(x):** נבצע תמיד $\text{Push}(x)$ למחסנית S_1 .
- פעולת Dequeue():**
 - אם המחסנית S_2 אינה ריקה, נבצע $\text{Pop}()$ מ- S_2 ונחזיר את האיבר.
 - אם המחסנית S_2 ריקה, נבצע "שפיכה" של כל האיברים מ- S_1 ל- S_2 (על ידי ביצוע $\text{Pop}()$ מ- S_1 ו- $\text{Push}()$ ל- S_2 לכל איבר). פעולה זו הופכת את סדר האיברים במדויק (מ-LIFO ל-FIFO).
 - לאחר השפיכה, נבצע $\text{Pop}()$ מ- S_2 . אם שתי המחסניות ריקות, נחזיר שגיאה.

ניתוח סיבוכיות לשיעורין

- במקרה הגרוע ביותר, פעולת Dequeue בודדת המאלצת שפיכה דורשת עבודה של $O(n)$.
ננתח את העלות לשיעורין באמצעות שיטת החיובים:
- נגדיר עלות משוערכת של $\hat{c} = 2$ עבור פעולת Enqueue, ועלות משוערכת של $\hat{c} = 0$ עבור פעולת Dequeue:
- בעת ביצוע $\text{Enqueue}(x)$: שקל אחד משמש להכנסה האמיתית ל- S_1 . שקל אחד מופקד כאשראי על גבי האיבר.
 - בעת ביצוע $\text{Dequeue}()$: המצריך שפיכה: העברת איבר מ- S_1 ל- S_2 (הכוללת Pop מ- S_1 ו- Push ל- S_2) עולה 2 פעולות אמיתיות. פעולות אלו ממומנות באופן מלא על ידי משיכת השקל שהופקד על גבי האיבר בעת כניסתו. המחיקה הסופית מ- S_2 ממומנת גם היא.
- מכיוון שכל איבר נכנס ל- S_1 פעם אחת, מועבר ל- S_2 פעם אחת, ונמחק מ- S_2 פעם אחת, האשראי המצטבר תמיד יכסה את עלויות השפיכה. העלות לשיעורין לכל פעולה היא $O(1)$.

6. מבוא לגרפים

הגדרות פורמליות

 גרף (G)

גרף הוא זוג סדור $G = (V, E)$, כאשר V היא קבוצה סופית שאינה ריקה של קודקודים (Vertices), ו- E היא קבוצה של קשתות (Edges) המקשרות בין זוגות קודקודים מ- V .

נסמן את מספר הקודקודים כ- $n = |V|$ ואת מספר הקשתות כ- $m = |E|$.

- גרף מכוון (Directed Graph):** הקשתות הן זוגות סדורים (u, v) המייצגים כיוון חד-סטרי מ- u ל- v .
- גרף לא מכוון (Undirected Graph):** הקשתות הן זוגות לא סדורים $\{u, v\}$, המייצגים קשר דו-כיווני.

מגבלות על מספר הקשתות m :

- בגרף פשוט לא מכוון ללא לולאות עצמיות: $0 \leq m \leq \frac{n(n-1)}{2} \in O(n^2)$.

- בגרף קשיר לא מכוון מתקיים בהכרח: $m \geq n - 1$. במצב זה, זמן ריצה של $O(n + m)$ שקול אסימפטוטית ל- $O(m)$.

דרכי ייצוג גרפים בזיכרון

1. מטריצת שכנות (Adjacency Matrix)

מערך דו-ממדי A בגודל $n \times n$. מתקיים $A[i][j] = 1$ אם קיימת קשת מהקודקוד i לקודקוד j , ו-0 אחרת.

- **סיבוכיות מקום:** $\Theta(n^2)$ ביטים.
- **בדיקת קיום קשת** (u, v) : $O(1)$ זמן.
- **מציאת כל השכנים של קודקוד** u : $\Theta(n)$ זמן (מאלץ סריקה של שורה שלמה במטריצה).

2. רשימת שכנות (Adjacency List)

מערך בגודל n של רשימות מקושרות, שבו התא i -י מכיל רשימה של כל הקודקודים שיש אליהם קשת מהקודקוד i .

- **סיבוכיות מקום:** $\Theta(n + m)$ (עבור קודקודים וקשתות). ייצוג זה חסכוני מאוד עבור גרפים דלילים (כאשר $m \ll n^2$).
- **בדיקת קיום קשת** (u, v) : $O(\deg(u))$ זמן (במקרה הגרוע ביותר $\Theta(n)$).
- **מציאת כל השכנים של קודקוד** u : $O(\deg(u))$ זמן (מעבר ישיר על הרשימה המקושרת).

7. עצים ועצי חיפוש בינאריים (BST)

מושגי יסוד בעצים

עץ הוא גרף קשיר ללא מעגלים. בעץ מושרש נבחר קודקוד אחד לשמש כשורש (Root).

- **אב (Parent) וכן (Child):** אם קיימת קשת בין u ל- v ו- u קרוב יותר לשורש, אזי u הוא האב של v , ו- v הוא הבן של u .
- **עלה (Leaf):** קודקוד שאין לו בנים כלל.
- **דרגה (Degree) של קודקוד:** מספר הבנים של הקודקוד.
- **עומק (Depth) של קודקוד** u : אורך המסלול היחיד מהשורש אל u (מספר הקשתות).
- **גובה (Height) של קודקוד** u : אורך המסלול הארוך ביותר מ- u לעלה כלשהו בצאצאיו. גובה העץ כולו הוא גובה השורש.
- **רמה (Level):** קבוצת כל הקודקודים בעלי עומק זהה. רמת השורש מוגדרת כ-0.

עץ בינארי

עץ שבו לכל קודקוד יש לכל היותר שני בנים, הנקראים בהתאמה "בן שמאלי" ו"בן ימני". סוגי עצים בינאריים:

- **עץ בינארי מלא (Full Binary Tree):** עץ שבו לכל קודקוד פנימי יש בדיוק שני בנים (אין קודקודים בעלי דרגה 1).
- **עץ בינארי שלם (Perfect Binary Tree):** עץ בינארי מלא שבו כל העלים נמצאים בדיוק באותה רמה (עומק זהה). מספר הקודקודים בעץ שלם בגובה h הוא בדיוק $n = 2^{h+1} - 1$, ומכאן שגובהו הוא $h = \Theta(\log n)$.
- **עץ בינארי כמעט שלם (Complete Binary Tree):** עץ שבו כל הרמות מלאות לחלוטין, פרט אולי לרמה האחרונה שממולאת ברציפות משמאל לימין ללא חורים. מאוזן תמיד ולכן גובהו $h \in O(\log n)$. ניתן לייצגו במערך ללא מצביעים בצורה הבאה עבור קודקוד באינדקס i :
 - הבן השמאלי: $\text{Left}(i) = 2i + 1$
 - הבן הימני: $\text{Right}(i) = 2i + 2$
 - האב: $\text{Parent}(i) = \lfloor \frac{i-1}{2} \rfloor$

סריקות בעצים בינאריים

1. סריקה לעומק (DFS)

מחולקת לשלושה סוגים בהתאם למועד הביקור בקודקוד הנוכחי ביחס לבניו (שמאל תמיד קודם לימין):

- **סריקה קדמית (Pre-order):** ביקור בקודקוד, סריקת תת-העץ השמאלי, סריקת תת-העץ הימני.
- **סריקה תוך-סדרית (In-order):** סריקת תת-העץ השמאלי, ביקור בקודקוד, סריקת תת-העץ הימני. בעץ חיפוש בינארי (BST), סריקה זו מחזירה את המפתחות בסדר ממוין עולה.
- **סריקה אחורית (Post-order):** סריקת תת-העץ השמאלי, סריקת תת-העץ הימני, ביקור בקודקוד.

2. סריקה לרוחב (BFS / Level-order)

ביקור בקודקודים לפי רמותיהם מלמעלה למטה ומשמאל לימין.

- **מימוש:** באמצעות **תור (Queue)**. נכניס את השורש לתור, ובכל שלב נוציא קודקוד, נבקר בו, ונכניס את בניו (שמאל ואז ימין) לתור.

שחזור עץ בינארי מסריקות

- **עץ בינארי רגיל:** ניתן לשחזר את מבנה העץ באופן יחיד על ידי שילוב של סריקת In-order ביחד עם סריקת Pre-order או Post-order. סריקת Pre-order או Post-order מספקת את זהות השורש של תת-העץ הנוכחי, וסריקת In-order מפרידה באופן ברור בין איברי תת-העץ השמאלי שלו (המופיעים משמאל לשורש בסריקה) לבין איברי תת-העץ הימני (המופיעים מימין).
- **שחזור משילוב Pre-order ו-Post-order:** אינו אפשרי באופן יחיד בעץ בינארי כללי (לדוגמה, לא ניתן להבחין בין בן שמאלי יחיד לבן ימני יחיד).
- **עץ חיפוש בינארי (BST):** ניתן לשחזר יחיד באמצעות סריקת Pre-order בלבד או Post-order בלבד, כיוון שסריקת ה-In-order שלו ידועה מראש (היא פשוט מיון המפתחות שקיבלנו).

עץ חיפוש בינארי (BST - Binary Search Tree)

עץ חיפוש בינארי (BST)

עץ בינארי המקיים את תכונת החיפוש הבינארי: לכל קודקוד x , כל המפתחות בתת-העץ השמאלי שלו קטנים ממפתח x , וכל המפתחות בתת-העץ הימני שלו גדולים ממפתח x .

פעולות יסוד ב-BST

1. חיפוש, הכנסה ומחיקה

כל הפעולות הללו דורשות מעבר לאורך מסלול יחיד בעץ, ולכן סיבוכיות הזמן שלהן במקרה הגרוע ביותר היא $O(h)$ (כאשר h הוא גובה העץ). במקרה הממוצע בעץ אקראי הגובה הוא $\Theta(\log n)$, ובמקרה הגרוע ביותר (עץ שנבנה כתוצאה מהכנסת איברים ממוינים והפך לשרשרת) הגובה הוא $\Theta(n)$.

2. מציאת עוקב (Successor) וקודם (Predecessor)

- **העוקב של קודקוד x :** האיבר בעל המפתח הקטן ביותר הגדול ממפתח x (הבא בתור בסדר ממוין).
- **מקרה א':** ל- x יש תת-עץ ימני. העוקב הוא האיבר המינימלי בתת-עץ זה (נלך פעם אחת ימינה, ואז שמאלה ככל הניתן).
- **מקרה ב':** ל- x אין תת-עץ ימני. נעלה כלפי מעלה לאבותיו של x עד אשר נגיע לפנייה הראשונה ימינה (כלומר, נמצא אב קדמון ש- x נמצא בתת-העץ השמאלי שלו).
- **הקודם של קודקוד x :** מוגדר באופן סימטרי (האיבר המקסימלי בתת-העץ השמאלי, או עלייה למעלה עד לפנייה שמאלה).

3. מחיקת קודקוד z

- **מקרה 1:** z הוא עלה: ננתק את המצביע אליו מאביו ונתיר את הזיכרון.
- **מקרה 2:** ל- z יש בן יחיד: נחבר את הבן של z ישירות לאביו של z (עקיפת הקודקוד).
- **מקרה 3:** ל- z יש שני בנים: נמצא את העוקב של z (נסמנו כ- y). מכיוון של- y יש שני בנים, הוא בהכרח נופל תחת מקרה 1 או 2 (אין לו בן שמאלי). נעתיק את תוכן המפתח של y לתוך z , ונמחק פיזית את הקודקוד y המקורי ממיקומו בעץ.

8. עצי חיפוש מאוזנים (AVL Trees)

עצי AVL

עץ AVL הוא עץ חיפוש בינארי המקיים את תנאי האיזון של AVL: לכל קודקוד v , הפרש הגבהים בין תת-העץ השמאלי שלו לתת-העץ הימני שלו הוא לכל היותר 1. נגדיר את גורם האיזון (Balance Factor):

$$BF(v) = h(v.\text{left}) - h(v.\text{right})$$

בעץ AVL מתקיים לכל קודקוד: $|BF(v)| \leq 1$.

משפט: חסם הגובה בעץ AVL

משפט: גובה עץ AVL

גובה עץ AVL בעל n קודקודים הוא תמיד $h = \Theta(\log n)$.

הוכחה:

נסמן ב- $N(h)$ את מספר הקודקודים המינימלי בעץ AVL שגובהו h . עץ כזה (הנקרא עץ פיבונאצ'י אסימפטוטי) מורכב משורש, תת-עץ אחד בגובה מינימלי $h-1$, ותת-עץ שני בגובה מינימלי $h-2$. נוסחת הנסיגה היא:

$$N(h) = 1 + N(h-1) + N(h-2)$$

נראה באינדוקציה כי $N(h) \geq 2^{h/2}$:

$$N(h) > N(h-1) + N(h-2) \geq 2^{\frac{h-1}{2}} + 2^{\frac{h-2}{2}} = 2^{\frac{h}{2}}(2^{-1/2} + 2^{-1}) \approx 2^{\frac{h}{2}}(0.7 + 0.5) > 2^{\frac{h}{2}}$$

מכיוון שמספר הקודקודים בפועל n מקיים $n \geq N(h) > 2^{h/2}$, נפעיל לוגריתם על שני האגפים:

$$\log n > \frac{h}{2} \implies h < 2 \log n \implies h \in O(\log n)$$

מאחר שלכל עץ בינארי מתקיים $h \in \Omega(\log n)$, נקבל חסם הדוק $h \in \Theta(\log n)$.

פעולות איזון ותיקון בעקבות שינויים

הכנסת איבר

הכנסת איבר מבוצעת תחילה כרגיל ב-BST, מה שעלול להגדיל את גובהו של תת-עץ ב-1 ולהפר את איזון ה-AVL של קודקודים לאורך מסלול ההכנסה אל השורש. נסרוק את המסלול מהקודקוד שהוכנס כלפי מעלה אל השורש. עבור הקודקוד הראשון שבו מופר האיזון ($|BF| = 2$), נבצע פעולת רוטציה (גלגול) מתאימה:

- **רוטציה פשוטה (Single Rotation - LL / RR):** מבוצעת כאשר הפרת האיזון היא "חד-כיוונית" (לדוגמה, הבר השמאלי והנכד השמאלי כבדים יותר). דורשת עדכון של 2 מצביעים ועלותה $O(1)$.
- **רוטציה כפולה (Double Rotation - LR / RL):** מבוצעת כאשר הפרת האיזון היא "זגזג" (לדוגמה, הבר השמאלי כבד והנכד הימני כבד). מבוצעת על ידי גלגול ראשון בין הבר לנכד, ולאחר מכן גלגול שני של האב. עלותה $O(1)$.

⚠ הערה: עצירת התיקון בהכנסה

בעת ביצוע רוטציה (פשוטה או כפולה) בעקבות הכנסת איבר, גובהו של תת-העץ המאוחר חוזר בדיוק לגובה שהיה לו לפני ההכנסה. לפיכך, גורמי האיזון של כל הצמתים מעליו במסלול אל השורש אינם משתנים, וניתן לעצור את הסריקה מיד. מובטח כי נדרש לכל היותר גלגול אחד (פשוט או כפול) לתיקון העץ בהכנסה. סיבוכיות כוללת: $O(\log n)$.

מחיקת איבר

נמחק את האיבר כרגיל ב-BST, ונעלה במסלול למעלה לכיוון השורש כדי לתקן את האיזון. בניגוד להכנסה, מחיקת איבר וביצוע רוטציה לתיקון קודקוד פנימי עלולים להקטין את גובהו של תת-העץ ב-1. הקטנה זו עלולה להפר את איזון ה-AVL של האב שמעליו, וכן הלאה. לכן, במחיקה **לא ניתן לעצור** לאחר גלגול יחיד, ויש להמשיך לבדוק ולתקן את גורמי האיזון לאורך כל המסלול עד להגעה לשורש. במקרה הגרוע ביותר, נבצע $O(\log n)$ רוטציות (לכל היותר רוטציה אחת בכל רמה של העץ). סיבוכיות כוללת: $O(\log n)$.

שימושים מיוחדים וחשובים מתקדמים ב-AVL

1. מציאת האיבר ה- k בגודלו בזמן $O(\log n)$

כדי לתמוך בשאילתה זו ביעילות, נוסיף לכל קודקוד v שדה עזר `size(v)` המייצג את מספר הקודקודים הכולל בתת-עץ ש- v הוא שורשו.

- **עדכון השדה:** בעת הכנסה או מחיקה, נעדכן את שדה ה-`size` לאורך מסלול הסריקה לשורש ב- $O(1)$ לכל קודקוד. בעת ביצוע רוטציה, נחשב מחדש את שדות ה-`size` של הקודקודים שהשתתפו בגלגול על פי בנייה ישירה של בניהם ב- $O(1)$.

אלגוריתם החיפוש `Select(root, k)`

```
def Select(node, k):
    if node is None or k > node.size:
        return Error
    left_size = node.left.size if node.left else 0
    if left_size == k - 1:
        return node
    elif left_size > k - 1:
        return Select(node.left, k)
    else:
        return Select(node.right, k - left_size - 1)
```

סיבוכיות זמן: $O(\log n)$ (מעבר לאורך מסלול יחיד בעץ).

2. יצירת עץ AVL ממערך ממוין בזמן $O(n)$

אם נתון מערך ממוין $A[0 \dots n-1]$, בניית עץ AVL על ידי הכנסת איברים בזה אחר זה תדרוש זמן של $O(n \log n)$. ניתן לייעל זאת ל- $O(n)$ באמצעות גישת הפרד ומשול הדומה לחיפוש בינארי: נבחר את החציון של המערך $A[\lfloor \frac{l+r}{2} \rfloor]$ לשמש כשורש העץ, ונבנה את תתי-העצים השמאלי והימני שלו באופן רקורסיבי על פי החצי השמאלי והחצי הימני של המערך בהתאמה. נוסחת הנסיגה לזמן הריצה היא:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1) \in O(n)$$

מכיוון שהחלוקה תמיד שווה, מובטח עץ מאוזן לחלוטין העונה על תנאי ה-AVL.

3. מיזוג שני עצי AVL בזמן אופטימלי

מקרה א': העצים מופרדים (כל המפתחות ב- T_1 קטנים מכל המפתחות ב- T_2)

נניח ללא הגבלת הכלליות כי $h(T_1) \geq h(T_2)$.

1. נשלוף את האיבר המקסימלי מ- T_1 (נסמנו כ- x) ב- $O(\log n)$ זמן על ידי ירידה ימינה. נמחק אותו פיזית מ- T_1 .
2. נרד לאורך המסלול הימני הקיצוני של T_1 עד אשר נמצא קודקוד u שגובהו שווה ל- $h(T_2) + 1$ או $h(T_2)$.
3. נחבר את תת-העץ המושרש ב- u ואת העץ T_2 כבנים של הקודקוד x החדש (כך ש- u הוא הבן השמאלי ו- T_2 הוא הבן הימני).
4. נתלה את x כבן הימני החדש של אביו של u .
5. נעלה למעלה לכיוון השורש ונתקן את איזון ה-AVL באמצעות רוטציות במידת הצורך. סיבוכיות זמן כוללת: $O(\log n)$ (שכן גובה העצים חסום לוגריתמית).

מקרה ב': העצים כלליים ומעורבבים

נמיר את שני העצים לרשימות ממוינות באמצעות סריקת In-order ב- $O(n)$. נמזג את שתי הרשימות הממוינות לרשימה אחת משותפת ב- $O(n)$ באמצעות שיטת המיזוג של Merge-Sort. נבנה עץ AVL חדש מהרשימה הממוזגת ב- $O(n)$ על פי האלגוריתם לעיל. סיבוכיות זמן כוללת: $O(n)$.

9. עצי חיפוש מסדר גבוה (B-Trees)

עץ m -ary הוא עץ חיפוש שבו לכל קודקוד פנימי יש לכל היותר m בנים ו- $m - 1$ מפתחות מסודרים.

עצי B-Tree

עץ B-Tree מדרגה m (כאשר m הוא לרוב מספר אי-זוגי הגדול מ-2) הוא עץ חיפוש מסדר גבוה המקיים את התכונות הבאות:

1. **שורש:** דרגת השורש היא לפחות 2 (אם אינו עלה) ולכל היותר m .
2. **דרגת קודקודים פנימיים:** כל קודקוד פנימי (פרט לשורש) מחזיק לכל היותר m בנים ולפחות $\lceil \frac{m}{2} \rceil$ בנים.
3. **מספר המפתחות:** כל קודקוד (פרט לשורש) מכיל לכל היותר $m - 1$ מפתחות ולפחות $\lceil \frac{m}{2} \rceil - 1$ מפתחות.
4. **רמת העלים:** כל העלים בעץ נמצאים בדיוק באותה רמה (עומק זהה).

גובה עץ B-Tree בעל n מפתחות הוא תמיד $h = \Theta(\log_m n)$. מבנה זה מאפשר ביצועים אופטימליים במערכות אחסון קבצים ובסיסי נתונים, שכן דרגת הפיצול הגבוהה מצמצמת את מספר הגישות לדיסק (I/O Operations) למינימום.

פעולות ב-B-Tree

הכנסת מפתח

נחפש את המקום המתאים להכנסת המפתח, שתמיד תתבצע בעלה.

- אם יש מקום בעלה (מספר המפתחות קטן מ- $m-1$): נכניס את המפתח במקומו הממוין ונסיים.
 - גלישה (Overflow): אם העלה כבר מלא ומכיל $m-1$ מפתחות, הכנסת האיבר החדש יוצרת מצב זמני של m מפתחות בקודקוד. נבצע פיצול (Split):
 1. נאתר את המפתח האמצעי (החציון מבין m האיברים).
 2. נפצל את שאר המפתחות לשני קודקודים חדשים בגודל $\frac{m-1}{2}$ מפתחות כל אחד.
 3. נעלה את החציון כלפי מעלה ונכניס אותו לקודקוד האב.
 4. במידה וגם האב מלא, נמשיך בביצוע הפיצול באופן רקורסיבי כלפי מעלה. אם הגענו לשורש והוא פוצל, ניצור שורש חדש המכיל את החציון בלבד. זהו המקרה היחיד שבו גובה עץ B-Tree גדל ב-1.
- סיבוכיות זמן: $O(m \log_m n)$ או $O(\log n)$ ללא תלות ישירה ב- m (קבוע).

מחיקת מפתח

מחיקת מפתח מבוצעת תמיד מעלה. אם המפתח המבוקש נמצא בקודקוד פנימי, נחליף אותו תחילה עם העוקב שלו (שבאחר ממצא בעלה) ולאחר מכן נמחק אותו מהעלה.

מחיקת מפתח מעלה עלולה לגרום למצב של חוסר (Underflow), שבו מספר המפתחות יורד מתחת למינימום הנדרש $1 - \lceil \frac{m}{2} \rceil$.

לתיקון חוסר נבחן את הצמתים השכנים מימין או משמאל בעלי אב משותף:

1. הלוואה משכן (Borrow): אם לאחד השכנים יש מפתחות עודפים (מעבר למינימום), נבצע "רוטציה" דרך האב: נעלה מפתח מהשכן לקודקוד האב, ונוריד את המפתח המתאים מהאב אל הקודקוד הבעייתי.
2. מיזוג צמתים (Merge): אם לכל השכנים יש רק את מספר המפתחות המינימלי, לא ניתן ללוות. נמזג את הקודקוד הבעייתי, את אחד השכנים שלו, ואת המפתח המפריד ביניהם מתוך האב לקודקוד אחד משותף. פעולה זו גורעת מפתח מהאב, ולכן עלולה לעורר חוסר באב, המצריך המשך תיקון רקורסיבי במעלה העץ. סיבוכיות זמן: $O(m \log_m n)$.

10. ערימות (Heaps)

ערימה בינארית היא עץ בינארי כמעט שלם המקיים את תכונת הערימה.

- ערימת מינימום (Min Heap): מפתח כל קודקוד קטן או שווה למפתחות של בניו. השורש מכיל תמיד את המפתח המינימלי בערימה.
- ערימת מקסימום (Max Heap): מפתח כל קודקוד גדול או שווה למפתחות של בניו. השורש מכיל תמיד את המפתח המקסימלי בערימה.

פעולות בערימה בינארית

1. Heapify-down (פעפוע למטה)

מיועדת לתיקון תכונת הערימה כאשר קודקוד מסוים מפר אותה מול בניו. נשווה את הקודקוד לבניו, ונחליף אותו עם הבן המינימלי (בערימת מינימום). נמשיך לבצע החלפות אלו כלפי מטה עד שהאיבר קטן מבניו או שהגענו לעלה.

סיבוכיות זמן: $O(h) = O(\log n)$.

2. Heapify-up (פעפוע למעלה)

מיועדת לתיקון תכונת הערימה כאשר איבר חדש שהוכנס מפר אותה מול אביו. נשווה את האיבר לאביו, ואם הוא קטן ממנו, נבצע החלפה ביניהם. נמשיך לעלות כלפי מעלה עד שהאיבר גדול מאביו או שהגענו לשורש. סיבוכיות זמן: $O(h) = O(\log n)$.

3. הכנסה ופעולת Delete-Min

- **הכנסת איבר:** נכניס את האיבר החדש במיקום הפנוי הבא במערך (העלה השמאלי ביותר ברמה התחתונה) כדי לשמור על עץ כמעט שלם, ונבצע Heapify-up לתיקון התכונה. סיבוכיות: $O(\log n)$.
- **הוצאת מינימום:** נחליף את ערך השורש עם האיבר האחרון בערימה (העלה הקיצוני ביותר), נגרע את האיבר האחרון מהערימה, ונבצע Heapify-down לשורש החדש כדי להחזיר את תכונת הערימה. סיבוכיות: $O(\log n)$.

בניית ערימה (Build-Heap) בזמן ליניארי $O(n)$

בהינתן מערך לא ממוין של n איברים, בניית ערימה על ידי הכנסת איברים בזה אחר זה תדרוש זמן של $O(n \log n)$. ניתן לייעל זאת ל- $O(n)$ באמצעות בנייה מלמטה למעלה: נתייחס לכל איברי החצי השני של המערך כערימות תקינות בגודל 1 (הרי עליהם מקיימים תמיד את תכונת הערימה באופן ריק). נסרוק את המערך מהאינדקס $1 - \lfloor \frac{n}{2} \rfloor$ ומטה לכיוון השורש (אינדקס 0), ונריץ את הפעולה Heapify-down עבור כל קודקוד.

הוכחה מתמטית ליעילות הבנייה

משפט: סיבוכיות בניית ערימה

זמן הריצה של אלגוריתם Build-Heap על מערך בגודל n הוא $O(n)$.

הוכחה:

מספר הקודקודים לכל היותר בגובה i בעץ בינארי כמעט שלם הוא $\lfloor \frac{n}{2^{i+1}} \rfloor$. פעולת Heapify-down על קודקוד בגובה i דורשת לכל היותר i צעדים. סך הכל צעדי הריצה חסומים על ידי:

$$T(n) \leq \sum_{i=1}^{\lfloor \log n \rfloor} \left\lfloor \frac{n}{2^{i+1}} \right\rfloor \cdot i \leq \frac{n}{2} \sum_{i=1}^{\infty} \frac{i}{2^i}$$

נחשב את הטור האינסופי $\sum_{i=1}^{\infty} \frac{i}{2^i}$. נשתמש בזהות הטור ההנדסי ל- $|x| < 1$:

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x}$$

נגזור את שני האגפים:

$$\sum_{i=1}^{\infty} i x^{i-1} = \frac{1}{(1-x)^2}$$

נכפול ב- x :

$$\sum_{i=1}^{\infty} ix^i = \frac{x}{(1-x)^2}$$

נציב $x = \frac{1}{2}$:

$$\sum_{i=1}^{\infty} \frac{i}{2^i} = \frac{1/2}{(1-1/2)^2} = \frac{1/2}{1/4} = 2$$

נציב בחזרה בחסם הכולל:

$$T(n) \leq \frac{n}{2} \cdot 2 = n \in O(n)$$

ערימת חציון (Median Heap)

מבנה נתונים התומך במציאת החציון של קבוצה דינמית של מספרים בזמן $O(1)$, ובהכנסה והסרה של איברים בזמן $O(\log n)$.

ארכיטקטורת המבנה

נחלק את n האיברים לשתי ערימות:

- ערימת מקסימום H_1** : מחזיקה את מחצית האיברים הקטנים ביותר בקבוצה ($\lfloor \frac{n}{2} \rfloor$ איברים). השורש של H_1 מייצג את האיבר המקסימלי מבין הקטנים.
- ערימת מינימום H_2** : מחזיקה את מחצית האיברים הגדולים ביותר בקבוצה ($\lceil \frac{n}{2} \rceil$ איברים). השורש של H_2 מייצג את האיבר המינימלי מבין הגדולים.

נשמור תמיד על תנאי האיזון הבא של גודל הערימות:

$$|H_2| - 1 \leq |H_1| \leq |H_2|$$

החציון יהיה תמיד השורש של ערימת המינימום H_2 (או ממוצע השורשים של H_1 ו- H_2 אם מספר האיברים הכולל זוגי).

- הכנסת איבר**: אם האיבר החדש קטן מהחציון, נכניס אותו ל- H_1 , ואם הוא גדול ממנו, נכניס אותו ל- H_2 . אם האיזון הופר, נבצע העברה של איבר השורש מהערימה הגדולה לערימה הקטנה ב- $O(\log n)$.

ערימה בינומית (Binomial Heap)

עץ בינומי (B_k)

עץ בינומי

עץ בינומי מסדר k , המסומן כ- B_k , הוא עץ סדור המוגדר רקורסיבית באופן הבא:

- העץ B_0 מורכב מקודקוד בודד.

- העץ B_k מורכב משני עצי B_{k-1} המחוברים יחד, כך ששורש העץ האחד הופך לבן השמאלי ביותר של שורש העץ השני.

תכונות עץ בינומי B_k (מוכחות באינדוקציה)

1. מספר הקודקודים בעץ הוא בדיוק 2^k .
2. גובה העץ הוא בדיוק k .
3. מספר הקודקודים בעומק i הוא בדיוק המקדם הבינומי $\binom{k}{i}$ (מכאן שמו).
4. דרגת השורש היא k , והבנים שלו הם שורשים של עצי בינומיים מסדר $B_0, B_1 \dots B_{k-1}$.

הגדרת ערימה בינומית

ערימה בינומית היא אוסף של עצי בינומיים המקיימים את תנאי הערימה (מפתח כל צומת קטן או שווה למפתחות בנוי), וכן את תנאי הדרגה המוגבלת: **קיים לכל היותר עץ בינומי אחד מכל דרגה**. מספר העצים בערימה בינומית בעלת n איברים חסום על ידי $\lfloor \log n \rfloor + 1$ (נגזר ישירות מהייצוג הבינארי של המספר n).

פעולות בערימה בינומית

1. פעולת Link (חיבור עצי בינומיים)

מקבלת שני שורשים של עצי בינומיים מאותה דרגה B_k , ומחברת אותם לעץ B_{k+1} יחיד בזמן $O(1)$, על ידי הצבת הקודקוד בעל המפתח הגדול יותר כבן של הקודקוד בעל המפתח הקטן יותר.

2. מיזוג ערימות (Merge)

הפעולה הבסיסית ביותר בערימות בינומיות. מבוצעת בדומה לחיבור שני מספרים בינאריים: אנו סורקים את העצים של שתי הערימות בסדר דרגות עולה. אם ישנם שני עצים מאותה דרגה, נחבר אותם באמצעות Link לעץ בדרגה הבאה (בדומה ל"שארית" בחיבור בינארי). סיבוכיות זמן: $O(\log n)$.

3. הכנסה והוצאת מינימום

- **הכנסת איבר:** ניצור ערימה בינומית חדשה המכילה איבר בודד, ונמזג אותה עם הערימה המקורית ב- $O(\log n)$. העלות לשיעורין של סדרת הכנסות היא $O(1)$ (בדומה לקידום מונה בינארי).
- **הוצאת מינימום:** נמצא את המינימום מבין שורשי העצים ב- $O(\log n)$. נסיר את השורש הזה. בנוי של השורש מהווים סדרה של עצי בינומיים תקינים. נהפוך את סדר העצים הללו ונתייחס אליהם כערימה בינומית חדשה, אותה נמזג עם הערימה המקורית ב- $O(\log n)$.

11. טבלאות גיבוב (Hash Tables)

טבלת גיבוב היא מבנה נתונים הממפה מפתחות מתוך יוניברס גדול U לתוך אינדקסים בטבלה בגודל m באמצעות פונקציית גיבוב $h : U \rightarrow \{0 \dots m - 1\}$. השאיפה היא שכל פעולות המילון (Insert, Delete, Search) יתבצעו ביעילות של $O(1)$.

מנגנוני גיבוב דטרמיניסטיים

בגיבוב דטרמיניסטי, פונקציית הגיבוב היא קבועה.

• **שיטת החילוק:** $h(k) = k \bmod m$.

- **בחירת m :** מומלץ לבחור את גודל הטבלה m כמספר ראשוני שאינו קרוב לחזקות של 2 או 10. אם נבחר $m = 2^p$, פונקציית הגיבוב תתחשב רק ב- p הביטים האחרונים של המפתח ותתעלם לחלוטין משאר המידע, מה שיגרום להתנגשויות רבות.

מנגנונים אקראיים ומשפחות אוניברסליות

כדי למנוע מצב שבו משתמש זדוני ייצר סדרת מפתחות הממפים כולם לאותו תא בטבלה ויהפוך את החיפוש לליניארי $O(n)$, נבחר את פונקציית הגיבוב באקראי מתוך משפחה מוגדרת.

משפחת פונקציות גיבוב אוניברסלית

משפחת פונקציות גיבוב $\mathcal{H}$ נקראת אוניברסלית אם לכל זוג מפתחות שונים $x \neq y \in U$, מספר הפונקציות $h \in \mathcal{H}$ עבורן מתקיים $h(x) = h(y)$ הוא לכל היותר $\frac{|\mathcal{H}|}{m}$. כלומר, בהגרלת פונקציה אקראית מ- $\mathcal{H}$, ההסתברות להתנגשות בין x ל- y היא לכל היותר $\frac{1}{m}$ (זוהי ההסתברות של פונקציית גיבוב אקראית מושלמת).

מימוש משפחה אוניברסלית מודולרית

נבחר מספר ראשוני גדול p המקיים $p \geq |U|$. לכל $a \in \{1 \dots p-1\}$ ו- $b \in \{0 \dots p-1\}$, נגדיר:

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$$

הקבוצה $\mathcal{H} = \{h_{a,b}\}$ מהווה משפחת גיבוב אוניברסלית. שמירת הבחירה דורשת אחסון של שני קבועים בלבד (a, b) בזיכרון, כלומר $O(1)$ מקום.

גיבוב מושלם דו-שלבי (Perfect Hashing)

גיבוב מושלם מיועד למילון סטטי (כאשר קבוצת המפתחות S ידועה מראש ואין הכנסות נוספות), ותומך בחיפוש בזמן של $O(1)$ במקרה הגרוע ביותר, תוך שימוש בנפח זיכרון כולל של $O(n)$.

ארכיטקטורת המבנה

הגיבוב מתבצע בשני שלבים:

1. **שלב ראשון:** נגדיר טבלה ראשית בגודל $m = n$, ונבחר פונקציית גיבוב אוניברסלית h הממפה את n האיברים לטבלה. נסמן ב- n_i את מספר המפתחות שגובבו לתא i בטבלה הראשית.
2. **שלב שני:** עבור כל תא i המכיל $n_i > 1$ איברים, נקים טבלת גיבוב משנית ייעודית בגודל $m_i = n_i^2$. נבחר עבורה פונקציית גיבוב אוניברסלית אישית h_i הממפה את n_i האיברים ללא אף התנגשות (על פי פרדוקס ימי ההולדת, בטבלה בגודל n^2 ההסתברות שאין התנגשויות כלל גדולה מחצי, ולכן הגרלה מהירה תניב פונקציה כזו בתוך מספר ניסיונות קבוע).

ניתוח נפח הזיכרון

נראה כי תוחלת נפח הזיכרון הכולל היא ליניארית $O(n)$:
 נפח הזיכרון הכולל נשלט על ידי סכום ריבועי מספר האיברים בתאים: $\sum_{i=0}^{n-1} n_i^2$.
 על פי תכונות משפחת גיבוב אוניברסלית, תוחלת סכום זה חסומה על ידי:

$$E \left[\sum_{i=0}^{n-1} n_i^2 \right] = n + 2 \cdot E[\text{Col}] < n + 2 \cdot \frac{n(n-1)}{2n} < 2n \in O(n)$$

מכאן שעלות המקום במקרה הגרוע ביותר היא $O(n)$, וזמן החיפוש מובטח להיות $O(1)$ במקרה הגרוע ביותר (שתי קריאות לפונקציות גיבוב פשוטות).

טיפול בהתנגשויות

1. שרשור (Chaining)

כל תא בטבלה $T[i]$ מכיל מצביע לרשימה מקושרת של כל האיברים שגובבו לאינדקס i .

• סיבוכיות זמן במקרה הממוצע:

- נגדיר את מקדם העומס $\alpha = \frac{n}{m}$. בהנחת גיבוב אחיד פשוט, תוחלת אורך כל רשימה מקושרת היא α .
- חיפוש לא מוצלח (האיבר אינו בטבלה): נסרוק את כל הרשימה, ולכן תוחלת הזמן היא $\Theta(1 + \alpha)$.
- חיפוש מוצלח: תוחלת הזמן היא $\Theta(1 + \frac{\alpha}{2})$.

- חסרונות:** בזבז מקום על מצביעים לרשימות, ופיזור האיברים בזיכרון הגורר פספוסים מטמון (Cache Misses) מרובים בשל חוסר לוקליות בזיכרון.

2. פנייה פתוחה (Open Addressing)

כל האיברים נשמרים בתוך הטבלה עצמה (לכן בהכרח $n \leq m$). פונקציית הגיבוב מקבלת פרמטר נוסף המייצג את מספר הניסיון: $h : U \times \{0 \dots m-1\} \rightarrow \{0 \dots m-1\}$.
 אם התא הראשון תפוס, נחשב אינדקסים חלופיים סדורים עד למציאת תא פנוי.

⚠ הערה: טיפול במחיקה בפנייה פתוחה

בעת מחיקת איבר מתא בפנייה פתוחה, לא ניתן לרוקן את התא ולהציב בו `Null`, שכן הדבר יקטע את שרשרת הניסיונות של איברים שהוכנסו אחריו וגובבו דרך תא זה, ויגרום לכך שהאלגוריתם לא ימצא אותם בעתיד. הפתרון הוא להציב בתא ערך מיוחד `Deleted` (או Tombstone), המאותת לאלגוריתם החיפוש להמשיך לסרוק דרכו, אך מאפשר לאלגוריתם ההכנסה לדרוס אותו ולרשום בו מפתח חדש.

שיטות סריקה נפוצות בפנייה פתוחה

1. בדיקה ליניארית (Linear Probing):

$$h(k, i) = (h'(k) + i) \bmod m$$

- **בעיית ההצטברות הראשונית (Primary Clustering):** גורמת ליצירת רצפים ארוכים של תאים תפוסים.

ככל שהרצף גדל, ההסתברות שאיבר חדש יפול לתוכו ויגדיל אותו גדלה באופן יחסי (הסתברות של $\frac{1+t}{m}$ במקום $\frac{1}{m}$), מה שפוגע קשות ביעילות החיפוש.

2. בדיקה ריבועית (Quadratic Probing):

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

מונעת הצטברות ראשונית, אך גוררת הצטברות משנית (איברים בעלי ערך גיבוב ראשוני זהה יסרקו בדיוק את אותו מסלול בבדיקה).

3. גיבוב כפול (Double Hashing):

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$$

אחת השיטות הטובות ביותר, שכן מסלול הבדיקה תלוי במפתח בשתי פונקציות שונות ומונע הצטברויות.

- **אילוץ על פונקציית הצעד $h_2(k)$:** כדי להבטיח שאלגוריתם הסריקה יסרוק את כל תאי הטבלה ולא יתקע בלולאה מחזורית חלקית, עלינו לוודא כי הצעד $h_2(k)$ זר לגודל הטבלה m לכל מפתח k (כלומר $\gcd(h_2(k), m) = 1$). דרך נפוצה להבטיח זאת היא לבחור את m כמספר ראשוני, ולהגדיר את פונקציית הצעד כמחזירה ערכים קטנים מ- m הגדולים מאפס (לדוגמה: $h_2(k) = 1 + (k \bmod (m - 1))$).

דוגמאות ריצה מלאות מתוך הרצאה 9

א. דוגמת ריצה: בדיקה ריבועית

נגדיר את פונקציית הבדיקה הריבועית הבאה על טבלה בגודל $m = 7$:

$$h(k, i) = (k \bmod 7 + i^2) \bmod 7$$

נכניס את המפתחות הבאים לטבלה ריקה בהתחלה:

1. הכנסת 76:

- $i = 0$: $h(76, 0) = (76 \bmod 7 + 0) \bmod 7 = (6 + 0) \bmod 7 = 6$. התא 6 פנוי, נכניס את 76 לשם.

2. הכנסת 40:

- $i = 0$: $h(40, 0) = (40 \bmod 7 + 0) \bmod 7 = (5 + 0) \bmod 7 = 5$. התא 5 פנוי, נכניס את 40 לשם.

3. הכנסת 48:

- $i = 0$: $h(48, 0) = (48 \bmod 7 + 0) \bmod 7 = 6$. תפוס על ידי 76 (התנגשות).
- $i = 1$: $h(48, 1) = (6 + 1^2) \bmod 7 = 0$. התא 0 פנוי, נכניס את 48 לשם.

4. הכנסת 5:

- $i = 0$: $h(5, 0) = 5$. תפוס על ידי 40 (התנגשות).
- $i = 1$: $h(5, 1) = (5 + 1^2) \bmod 7 = 6$. תפוס על ידי 76 (התנגשות).
- $i = 2$: $h(5, 2) = (5 + 2^2) \bmod 7 = 9 \bmod 7 = 2$. התא 2 פנוי, נכניס את 5 לשם.

5. הכנסת 55:

- $i = 0$: $h(55, 0) = 6$. תפוס על ידי 76.
- $i = 1$: $h(55, 1) = 0$. תפוס על ידי 48.
- $i = 2$: $h(55, 2) = (6 + 2^2) \bmod 7 = 3$. התא 3 פנוי, נכניס את 55 לשם.

מצב הטבלה הסופי:

- תא 0: 48, תא 1: NIL, תא 2: 5, תא 3: 55, תא 4: NIL, תא 5: 40, תא 6: 76.

ב. דוגמת ריצה: גיבוב כפול

נגדיר את שתי פונקציות הגיבוב הבאות על טבלה בגודל $m = 13$:

$$h_1(k) = k \bmod 13$$

$$h_2(k) = 1 + (k \bmod 11)$$

פונקציית הערבול הכללית היא: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 13$.

נניח שבטבלה כבר מאוחסנים האיברים הבאים: תא 1 מכיל 79, תא 4 מכיל 69, תא 5 מכיל 98, תא 7 מכיל 72, תא 11 מכיל 50.

כעת נכניס את המפתח $k = 14$:

- נחשב את ערכי הפונקציות עבור 14:

$$h_1(14) = 14 \bmod 13 = 1$$

$$h_2(14) = 1 + (14 \bmod 11) = 4$$

- **ניסיון 0** ($i = 0$): $h(14, 0) = 1$. תא 1 תפוס על ידי 79 (התנגשות).
- **ניסיון 1** ($i = 1$): $h(14, 1) = (1 + 1 \cdot 4) \bmod 13 = 5$. תא 5 תפוס על ידי 98 (התנגשות).
- **ניסיון 2** ($i = 2$): $h(14, 2) = (1 + 2 \cdot 4) \bmod 13 = 9$. תא 9 פנוי, נכניס את 14 לשם.

מצב הטבלה לאחר ההכנסה:

- תא 1: 79, תא 4: 69, תא 5: 98, תא 7: 72, תא 9: 14, תא 11: 50, כל השאר NIL.

ניתוח סיבוכיות מתמטי מפורט בפנייה פתוחה

תחת הנחת הערבול האחיד (Uniform Hashing), שבה כל מפתח מוגרל באופן שווה להיות כל אחת מ- m סדרות הבדיקה האפשריות:

1. חיפוש לא מוצלח והכנסה**משפט: תוחלת מספר הבדיקות בחיפוש לא מוצלח**

בטבלת גיבוב עם גורם עומס $\alpha = n/m < 1$, תוחלת מספר הבדיקות בחיפוש לא מוצלח (או בהכנסה) חסומה מלעיל על ידי:

$$\frac{1}{1 - \alpha}$$

הוכחה:

יהי X משתנה מקרי המייצג את מספר הבדיקות המבוצעות בחיפוש לא מוצלח. האירוע שבו אנו זקוקים ללפחות i בדיקות ($X \geq i$) מתקיים אם ורק אם $i - 1$ הבדיקות הראשונות שביצענו הגיעו כולן לתאים תפוסים.

נסמן ב- A_j את האירוע שבו הבדיקה ה- j מצאה תא תפוס. לכן:

$$P(X \geq i) = P(A_1 \cap A_2 \cap \dots \cap A_{i-1}) = P(A_1) \cdot P(A_2 | A_1) \cdots P(A_{i-1} | A_1 \cap \dots \cap A_{i-2})$$

נחשב את ההסתברויות המותנות:

- בבדיקה 1, ישנם n תאים תפוסים מתוך m תאים סה"כ בטבלה: $P(A_1) = \frac{n}{m}$.
 - בבדיקה 2, בהינתן שהראשונה פגשה תא תפוס, נותרו $m - 1$ תאים מתוכם $n - 1$ תפוסים: $P(A_2 | A_1) = \frac{n-1}{m-1}$.
 - בבדיקה j , בהינתן שכל הקודמות פגשו תאים תפוסים: $P(A_j | A_1 \cap \dots \cap A_{j-1}) = \frac{n-j+1}{m-j+1}$.
- מאחר ש- $n < m$, לכל $j \geq 1$ מתקיים $\frac{n-j+1}{m-j+1} \leq \frac{n}{m} = \alpha$. מכאן שנוכל לחסום את ההסתברות המשותפת:

$$P(X \geq i) \leq \alpha \cdot \alpha \dots \alpha = \alpha^{i-1}$$

נחשב את התוחלת $E[X]$ באמצעות סכום הסתברויות הזנב:

$$E[X] = \sum_{i=1}^{\infty} P(X \geq i) \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}$$

ההוכחה הושלמה.

2. חיפוש מוצלח

משפט: תוחלת מספר הבדיקות בחיפוש מוצלח

בטבלת גיבוב עם גורם עומס $\alpha = n/m < 1$, תוחלת מספר הבדיקות בחיפוש מוצלח חסומה מלעיל על ידי:

$$\frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$$

הוכחה:

חיפוש מוצלח של מפתח k עוקב בדיוק אחר אותו מסלול בדיקות שבוצע כאשר האיבר הוכנס לטבלה לראשונה. אם k היה האיבר ה- $i+1$ שהוכנס לטבלה, מספר הבדיקות הצפוי למציאתו זהה למספר הבדיקות הצפוי להכנסתו לטבלה המכילה i איברים, שחסום לפי המשפט הקודם על ידי:

$$\frac{1}{1-i/m} = \frac{m}{m-i}$$

נחשב את ממוצע זמני ההכנסה הללו על פני כל n האיברים הקיימים בטבלה:

$$E[\text{Search}] = \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m-i} = \frac{1}{\alpha} \sum_{i=0}^{n-1} \frac{1}{m-i}$$

נשים לב שהסכום $\sum_{i=0}^{n-1} \frac{1}{m-i} = \frac{1}{m} + \frac{1}{m-1} + \dots + \frac{1}{m-n+1} = \sum_{k=m-n+1}^m \frac{1}{k}$ נבצע קירוב אינטגרלי לסכום הטור ההרמוני החלקי באמצעות השטח מתחת לפונקציה $f(x) = 1/x$:

$$\sum_{k=m-n+1}^m \frac{1}{k} \leq \int_{m-n}^m \frac{1}{x} dx = [\ln(x)]_{m-n}^m = \ln(m) - \ln(m-n) = \ln \left(\frac{m}{m-n} \right)$$

מאחר ש- $\frac{m-n}{m} = 1 - \alpha$, הביטוי הופך ל- $\ln \left(\frac{1}{1-\alpha} \right)$.

נציב בחזרה במשוואת התוחלת ונקבל:

$$E[\text{Search}] \leq \frac{1}{\alpha} \ln \left(\frac{1}{1 - \alpha} \right)$$

ההוכחה הושלמה.

3. גיבוב קוקייה (Cuckoo Hashing)

גיבוב קוקייה הוא מבנה נתונים ייחודי ומתקדם למימוש מילון, המבטיח זמן ביצוע של $O(1)$ במקרה הגרוע ביותר (Worst-Case) לפעולות חיפוש ומחיקה.

ארכיטקטורת המבנה

- **שתי טבלאות גיבוב:** אנו מחזיקים שתי טבלאות נפרדות, T_1 ו- T_2 , בגודל שווה m כל אחת.
- **שתי פונקציות גיבוב עצמאיות:** אנו משתמשים בשתי פונקציות גיבוב עצמאיות h_1, h_2 הממפות מהיוניברס לתחומי האינדקסים של הטבלאות.
- **שני מיקומים אפשריים בלבד:** כל מפתח $x \in U$ יכול להימצא אך ורק באחד משני התאים הבאים:
 - בטבלה הראשונה במיקום $T_1[h_1(x)]$
 - בטבלה השנייה במיקום $T_2[h_2(x)]$

פעולות המבנה

1. **חיפוש (Lookup(x)):** בודקים את שני המיקומים האפשריים. אם המפתח קיים, הוא חייב להיות באחד מהם.
 - **סיבוכיות זמן:** $O(1)$ במקרה הגרוע ביותר (קבוע קשיח של לכל היותר שתי בדיקות).
2. **מחיקה (Delete(x)):** מחפשים את המפתח בשני התאים. אם הוא נמצא, מוחקים אותו ומציבים NIL.
 - **סיבוכיות זמן:** $O(1)$ במקרה הגרוע ביותר.
3. **הכנסה (Insert(x)):**
 - אנו מנסים להציב את x ב- $T_1[h_1(x)]$.
 - אם התא פנוי, אנו מסיימים.
 - אם התא תפוס על ידי מפתח קיים y , אנו **בועטים** את y החוצה ומציבים את x . כעת אנו חייבים למצוא מקום ל- y .
 - אנו מנסים להציב את y בטבלה השנייה $T_2[h_2(y)]$. אם תא זה תפוס על ידי z , אנו בועטים את z ומנסים לשכן אותו ב- $T_1[h_1(z)]$, וחוזר חלילה.
 - **טיפול בלולאה ורה-האש:** ייתכן ששרשרת הבעיטות תיכנס ללולאה אינסופית. אנו מגדירים חסם עליון למספר הבעיטות המותרות $MaxLoop = c \log n$. אם הגענו ל- $MaxLoop$ בעיטות, אנו עוצרים, מגרילים פונקציות גיבוב חדשות ומבצעים **בנייה מחדש (Rehash)** לכל האיברים.

⚠️ זהירות - מגבלת גורם העומס בגיבוב קוקייה

כדי להבטיח את יעילות המבנה ותדירות Rehash נמוכה, גורם העומס $\alpha = \frac{n}{2m}$ חייב להיות קטן מ-0.5 ($\alpha < 0.5$). אם נפתח מעבר ל-50% מקיבולת הטבלאות, ייווצרו לולאות בעיטות אינסופיות בתדירות גבוהה והמבנה יקרוס ביעילותו.

גרף קוקייה (Cuckoo Graph)

גרף קוקייה הוא כלי מתמטי לניתוח שרשרת הבעיטות.

נגדיר גרף דו-צדדי לא מכוון $G = (V, E)$:

- **קבוצת הקודקודים** V : מייצגים את כל $2m$ התאים של שתי הטבלאות.
- **קבוצת הקשתות** E : לכל מפתח x השמור בטבלה, קיימת קשת המקשרת בין התא $h_1(x) \in T_1$ לבין $h_2(x) \in T_2$.

הכנסת מפתח חדש x שקולה להוספת קשת חדשה $e = (h_1(x), h_2(x))$ לגרף. ננתח את סוג רכיב הקשירות V_c שאליו מתווספת הקשת החדשה:

- **עץ (Tree)**: הרכיב אינו מכיל מעגלים. הוספת הקשת יוצרת מעגל יחיד בודד. שרשרת הבעיטות תנוע לאורך הגרף ותיעצר בהכרח כאשר נגיע לקודקוד עלה פנוי (תא פנוי). ההכנסה מסתיימת בהצלחה.
- **מעגל יחיד (Single Cycle)**: הרכיב הכיל מעגל אחד לפני ההכנסה. הוספת הקשת יוצרת שני מעגלים. שרשרת הבעיטות תנוע לאורך המעגל, תחזור על עקבותיה, אך תצליח למצוא קודקוד עלה פנוי מחוץ למעגלים וההכנסה תסתיים בהצלחה.
- **מעגל כפול (Double Cycle) ומעלה**: מספר הקשתות ברכיב גדול או שווה למספר הקודקודים בתוספת 1 ($|E_c| \geq |V_c| + 1$). על פי עקרון שובר היונים, לא ניתן לשכן $|V_c| + 1$ מפתחות (קשתות) בתוך $|V_c|$ תאים (קודקודים) כך שכל מפתח יתפוס תא ייחודי. שרשרת הבעיטות תיכנס ללולאה אינסופית ותגרור ביצוע **Rehash**.

הוכחת תוחלת זמן הכנסה $O(1)$

משפט: תוחלת זמן ההכנסה בגיבוב קוקייה

תחת ההנחה שגורם העומס $\alpha \leq \frac{1}{2} - \varepsilon$ עבור קבוע $\varepsilon > 0$, תוחלת זמן ההכנסה של מפתח היא $E[T] = O(1)$.

הוכחה:

נחלק את זמן הריצה של ההכנסה לשני חלקים:

1. **זמן בעיטות (Time of kicks)**: מספר הבעיטות המבוצעות (עד $MaxLoop = c \log n$). תחת ההנחה ש- $\alpha < 0.5$, ההסתברות למסלול בעיטות באורך k דועכת מעריכית ב- k (כמו טור הנדסי מתכנס), ולכן תוחלת מספר הבעיטות היא קבועה: $E[Kicks] = O(1)$.
2. **זמן בנייה מחדש (Rehash)**: אם נקלענו ללולאה או עברנו את מגבלת הצעדים, אנו מבצעים **Rehash** בעלות של $O(n)$ לכל האיברים. ההסתברות להתרחשות **Rehash** (קיום מעגל כפול או מסלול ארוך מ- $c \log n$) חסומה על ידי $P(\text{Rehash}) \leq \frac{d}{n^2} \in O(1/n)$ עבור קבוע d מתאים.

נחשב את תוחלת זמן ההכנסה הכולל:

$$E[T] = E[Kicks] + P(\text{Rehash}) \times \text{Cost of Rehash} \leq O(1) + O\left(\frac{1}{n}\right) \times O(n) = O(1) + O(1) = O(1)$$

הוכחה הושלמה.

12. קבוצות זרות (Union-Find / Disjoint Sets)

מבנה נתונים המנהל אוסף של קבוצות זרות $S_1, S_2 \dots S_k$. לכל קבוצה יש איבר נציג ייחודי המשמש כשם הקבוצה.

• פעולות נתמכות:

- $\text{MakeSet}(x)$: יצירת קבוצה חדשה המכילה את האיבר הבדוד x .
- $\text{Find}(x)$: מציאת נציג הקבוצה שאליו שייך האיבר x . משמש לקביעה האם שני איברים שייכים לאותה קבוצה.
- $\text{Union}(x, y)$: איחוד שתי הקבוצות המיוצגות על ידי הנציגים x ו- y לקבוצה אחת משותפת.

דרכי מימוש ויעילות

1. מימוש באמצעות רשימות מקושרות

כל קבוצה מיוצגת כרשימה מקושרת שבה כל איבר מכיל מצביע ישיר לראש הרשימה (הנציג).

- עלות של $O(1)$ זמן (גישה ישירה דרך המצביע לראש) : $\text{Find}(x)$.
- נצטרך לעדכן את כל המצביעים של איברי הרשימה השנייה להצביע על ראש הרשימה : $\text{Union}(x, y)$. $O(n)$ הראשונה. במקרה הגרוע, פעולה בודדת תעלה.

שיפור: איחוד לפי גודל (Weighted Union)

בעת איחוד שתי הקבוצות, נשמור שדה גודל בראש כל רשימה, ונחבר תמיד את הקבוצה הקטנה יותר לסופה של הקבוצה הגדולה יותר (נעדכן את המצביעים של איברי הקבוצה הקטנה).

✓ טענה: חסם עדכון מצביעים ב-Weighted Union

תחת שיפור האיחוד לפי גודל, המצביע לראש הקבוצה של איבר כלשהו x משתנה לכל היותר $\lfloor \log n \rfloor$ פעמים לאורך סדרת פעולות.

הוכחה:

בכל פעם שהמצביע של x מעודכן להצביע על נציג חדש, פירוש הדבר הוא שהקבוצה של x אוחדה לתוך קבוצה הגדולה או שווה לה בגודלה. לפיכך, גודל הקבוצה החדשה של x הוא לפחות פי 2 מגודלה הקודם. מכיוון שגודל הקבוצה המקסימלי חסום על ידי n , מספר ההכפלות של גודל הקבוצה מוגבל לכל היותר $\log n$ פעמים. מכאן שסך כל הזמן הנדרש לביצוע m פעולות איחוד וחיפוש הוא $O(m + n \log n)$, מה שמניב עלות משוערכת לשיעורין של $O(\log n)$ לפעולה.

2. מימוש באמצעות יער של עצים הפוכים (Forest of Trees)

כל קבוצה מיוצגת כעץ שבו כל קודקוד מכיל מצביע אך ורק לאביו (parent). שורש העץ הוא נציג הקבוצה.

• פעולות בסיסיות:

- עלייה במעלה העץ באמצעות מצביעי האב עד להגעה לשורש : $\text{Find}(x)$.
- תליית שורש עץ אחד תחת שורש העץ השני (עדכון מצביע האב של השורש) : $\text{Union}(x, y)$.

שיפורים מתקדמים להשגת זמן כמעט ליניארי

כדי למנוע מהעצים להפוך לשרשראות ארוכות, נפעיל שני שיפורים במקביל:

1. **איחוד לפי דרגה (Union by Rank):** נשמור שדה `rank` לכל שורש המייצג חסם עליון לגובה העץ. בעת ביצוע איחוד, נתלה תמיד את השורש בעל הדרגה הנמוכה יותר תחת השורש בעל הדרגה הגבוהה יותר. הדרגה גדלה ב-1 רק כאשר אנו מאחדים שני עצים בעלי דרגה זהה. שיפור זה מבטיח כי גובה כל עץ אינו עולה על $\log n$.
2. **כיווץ מסלולים (Path Compression):** במהלך ביצוע פעולת `Find(x)`, לאחר שמצאנו את השורש `r`, נבצע סבב עדכון שני שבו נשנה את מצביעי האב של כל הקודקודים שעברנו בהם לאורך מסלול החיפוש מ-`x` ועד השורש, כך שיצביעו ישירות על השורש `r`. פעולה זו "משטחת" את העץ באופן משמעותי ומייעלת קריאות עתידיות.

דוגמת מעקב שלב אחר שלב (איחוד לפי דרגה וכיווץ מסלולים)

נניח שישנם 8 איברים במערכת, המאותחלים ב- `Make-Set`: $\{1, 2, 3, 4, 5, 6, 7, 8\}$.
בתחילה נוצרים 8 עצים נפרדים בגובה 0: לכל איבר `i` `parent[i] = i` ו-`rank[i] = 0`.

שלב 1: איחודים ראשוניים

נבצע סדרת איחודים: `Union(1, 2)`, `Union(3, 4)`, `Union(5, 6)`, `Union(7, 8)`.
בכל זוג הדרגות שוות ל-0, לכן נחבר אחד לשני ונגדיל את דרגת האב ל-1:

- `parent[1] = 2`, `rank[2] = 1`.
 - `parent[3] = 4`, `rank[4] = 1`.
 - `parent[5] = 6`, `rank[6] = 1`.
 - `parent[7] = 8`, `rank[8] = 1`.
- נוצרו 4 עצים בעלי גובה 1 ודרגה 1.

שלב 2: איחודים משניים

נבצע `Union(1, 3)`:

- `Find-Set(1)` (בעל דרגה 1) עולה לאב ומחזיר שורש 2.
- `Find-Set(3)` (בעל דרגה 1) עולה לאב ומחזיר שורש 4.
- מאחר והדרגות שוות, נחבר את 2 להצביע על 4. `parent[2] = 4` והדרגה של 4 גדלה: `rank[4] = 2`.

נבצע `Union(5, 7)`:

- `Find-Set(5)` (דרגה 1) מחזיר שורש 6.
- `Find-Set(7)` (דרגה 1) מחזיר שורש 8.
- נחבר את 6 להצביע על 8. `parent[6] = 8` והדרגה של 8 גדלה: `rank[8] = 2`.
נוצרו 2 עצים בעלי דרגה 2.

שלב 3: איחוד שלישי

נבצע `Union(1, 5)`:

- `Find-Set(1)` (דרגה 2) עולה דרך 2 לשורש 4 (דרגה 2).
- `Find-Set(5)` (דרגה 2) עולה דרך 6 לשורש 8 (דרגה 2).

- הדרגות שוות, ולכן נחבר את 4 להצביע על 8. $\text{parent}[4] = 8$ והדרגה של 8 גדלה: $\text{rank}[8] = 3$.
העץ המאוחד הנוכחי מכיל את כל 8 האיברים כאשר השורש הראשי הוא 8 (בעל דרגה 3).

שלב 4: ביצוע $\text{Find-Set}(1)$ עם כיווץ מסלולים

כעת אנו קוראים ל- $\text{Find-Set}(1)$ במבנה שנוצר.

- מסלול החיפוש מ-1 לשורש 8 הוא: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8$.
- במהלך החזרה מהרקורסיה של ה- Find-Set , כל הקודקודים לאורך המסלול מעודכנים להצביע ישירות על השורש 8:
- $\text{parent}[1] = 8$
- $\text{parent}[2] = 8$
- $\text{parent}[4] = 8$
- **מצב הדרגות בסיום:** הדרגות של הקודקודים בעץ אינן משתנות עקב כיווץ מסלולים, ולכן הן נותרות: $\text{rank}[8]$
 $\text{rank}[6] = 1$, $\text{rank}[2] = 1$, $\text{rank}[4] = 2$, $\text{rank}[3] = 3$. עם זאת, גובהו של העץ בפועל התכווץ ל-2 בלבד!

הוכחת חסם הריצה $O(m \log^* n)$ בשיטת החיובים (Accounting Method)

הגדרת פונקציית לוג כוכבית (Iterated Logarithm)

הפונקציה $\log^* n$ מוגדרת כמספר הפעמים שיש להפעיל את פונקציית הלוגריתם בבסיס 2 על המספר n כדי לקבל ערך שקטן או שווה ל-1:

$$\log^* n = \begin{cases} 0 & n \leq 1 \\ 1 + \log^*(\log n) & n > 1 \end{cases}$$

פונקציה זו גדלה בקצב איטי בצורה קיצונית. לכל ערך מעשי או פיזיקלי מתקיים $\log^* n \leq 5$ (לדוגמה, $\log^*(2^{65536}) = 5$).

הוכחת המשפט

נחלק את ציר הדרגות האפשריות של הקודקודים במבנה (מ-0 עד $\log n$) לתת-קבוצות הנקראות **גושים** (Blocks).
גוש t (עבור $t \geq 0$) מוגדר על ידי הטווח הבא:

$$B_t = \{F(t-1) + 1, \dots, F(t)\}$$

כאשר הפונקציה F מוגדרת רקורסיבית כמגדל חזקות של 2:

$$F(t) = 2^{F(t-1)} \quad \text{רובע} \quad F(-1) = 0$$

הגושים הראשונים הם:

- גוש 0: $\{0\}$ (מכיוון ש- $F(-1) = 0, F(0) = 2^0 = 1$)
- גוש 1: $\{1\}$
- גוש 2: $\{2\}$ (מכיוון ש- $F(1) = 2$)
- גוש 3: $\{3, 4\}$ (מכיוון ש- $F(2) = 4$)
- גוש 4: $\{5, \dots, 16\}$ (מכיוון ש- $F(3) = 16$)
- גוש 5: $\{17, \dots, 65536\}$ (מכיוון ש- $F(4) = 65536$)

מאחר שהדרגה המקסימלית של קודקוד בעץ היא לכל היותר $\log_2 n$, מספר הגושים הכולל המכסה את כל הדרגות הוא לכל היותר $\log^* n$.

1. שלב ההפקדה בבנק (Make-Set)

בעת יצירת איבר x , אנו מפקידים עבורו מטבעות בבנק המבנה. אם דרגתו הסופית של x שייכת לגוש t (כלומר $\text{rank}[x] \in B_t$), אנו נותנים לו $F(t)$ מטבעות.

✓ **טענה: סך המטבעות שהופקדו בבנק חסום על ידי $O(n \log^* n)$**

הוכחה:

מספר הקודקודים שדרגתם היא k חסום על ידי $n/2^k$. עבור גוש t מסוים, מספר הקודקודים שדרגתם הסופית נמצאת בטווח הגוש B_t הוא לכל היותר:

$$\sum_{k=F(t-1)+1}^{F(t)} \frac{n}{2^k} \leq \frac{n}{2^{F(t-1)}} = \frac{n}{F(t)}$$

כל קודקוד כזה מקבל $F(t)$ מטבעות. לכן, סך כל המטבעות שחולקו לקודקודים בגוש t חסום על ידי:

$$\frac{n}{F(t)} \cdot F(t) = n$$

מאחר שישנם לכל היותר $\log^* n$ גושים, סך כל המטבעות שהופקדו בריצה כולה הוא לכל היותר $O(n \log^* n)$.

2. שלב התשלום על פעולות Find-Set

נבחן קריאה ל- $\text{Find-Set}(x)$. הפעולה עוברת על מסלול קודקודים $x = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_r = \text{root}$. אנו מחלקים את הקודקודים במסלול לשלוש קטגוריות שמשלמות על הביקור בהם:

1. **השורש v_r ובנו הישיר v_{r-1} :** הביקור בהם עולה לכל היותר 2 צעדים. פעולת ה- Find-Set משלמת עליהם ישירות (עלות קבועה $O(1)$).
2. **קודקודים v_i שדרגתם ודרגת אביהם נמצאות בגושים שונים:** מכיוון שדרגות הקודקודים עולות ממש לאורך המסלול, ומספר הגושים הכולל חסום על ידי $\log^* n$, יכולים להיות לכל היותר $\log^* n$ קודקודים כאלו לאורך המסלול. פעולת ה- Find-Set משלמת עליהם ישירות (עלות $O(\log^* n)$).
3. **קודקודים v_i שדרגתם ודרגת אביהם נמצאות באותו הגוש t :** הביקור בהם משולם מתוך מטבעות הפיקדון של הקודקוד עצמו v_i (מטבע 1 לכל ביקור).

נראה שקודקוד מהסוג השלישי לעולם לא יחרוג מתקציב המטבעות שלו:

בכל פעם שאנו מבצעים Find-Set ועוברים על קודקוד v_i ששייך לגוש t יחד עם אביו, כתוצאה מכיוון מסלולים האב החדש של v_i יהיה השורש v_r .

מכיוון שדרגת השורש גדולה ממש מהדרגה של האב הקודם, דרגת האב החדש של v_i גדולה ממש מדרגת האב הקודם שלו.

הקודקוד v_i יכול לשלם מהבנק שלו לכל היותר עד שדרגת אביו תצא מגוש t . מכיוון שדרגת האב גדלה בכל תשלום כזה, ומאחר שהפרש הדרגות המקסימלי בגוש t הוא $F(t)$, הקודקוד v_i ישלם מהפיקדון שלו לכל היותר $F(t)$ פעמים לאורך כל הריצה.

מכיוון שהפקדנו עבורו בדיוק $F(t)$ מטבעות ב- Make-Set , מובטח כי תקציבו לא ייגמר לעולם.

סיכום הניתוח לשיעור

במהלך m פעולות במבנה נתונים:

- כל פעולת Find-Set משלמת ישירות $O(1)$ עבור השורש ובנו, ו- $O(\log^* n)$ עבור קודקודים במעבר בין גושים. סך הכל: $O(m \log^* n)$.
- כל שאר העלויות משולמות מהפיקדונות שהפקדנו בבנק, שסכומם הכולל חסום על ידי $O(n \log^* n)$. מכיוון ש- $m \geq n$ (שכן כל איבר חייב להיווצר על ידי פעולת Make-Set), סך כל זמן הריצה של סדרת הפעולות חסום על ידי:

$$O(m \log^* n)$$

הוכחנו את המשפט.

משפט: סיבוכיות Union-Find אופטימלי (פונקציית אקרמן ההפוכה)

ניתוח מתמטי מדויק ומעמיק יותר (שהוצג על ידי רוברט טרג'אן) מוכיח כי השילוב של איחוד לפי דרגה וכיווץ מסלולים מניב זמן ריצה כולל של $O(m \cdot \alpha(n))$, כאשר $\alpha(n)$ היא פונקציית אקרמן ההפוכה (Inverse Ackermann Function). פונקציה זו גדלה בקצב איטי להפליא, ולכל צורך מעשי חסומה על ידי 4.

13. פרדיגמות עיצוב אלגוריתמים

הפרד ומשול (Divide & Conquer)

פרדיגמה זו מבוססת על שלושה שלבים:

1. הפרד: חלוקת הבעיה למספר תת-בעיות זרות וקטנות יותר מאותו סוג.
2. משול: פתרון תת-הבעיות באופן רקורסיבי.
3. שלב: שילוב פתרונות תת-הבעיות לקבלת הפתרון לבעיה המקורית.

דוגמאות בולטות

1. כפל מחרוזות ביטים מהיר (Karatsuba)

כפל סטנדרטי של שני מספרים בגודל n ביטים דורש $O(n^2)$ פעולות בינאריות. נחלק כל מספר x, y לשני חצאים בגודל $\frac{n}{2}$ ביטים:

$$x = x_1 \cdot 2^{n/2} + x_2$$

$$y = y_1 \cdot 2^{n/2} + y_2$$

מכפלת המספרים היא:

$$xy = x_1 y_1 \cdot 2^n + (x_1 y_2 + x_2 y_1) \cdot 2^{n/2} + x_2 y_2$$

חישוב ישיר בצורה זו דורש 4 מכפלות של מספרים בגודל $\frac{n}{2}$, מה שמניב רקורסיה של $O(n^2) \implies T(n) = 4T(\frac{n}{2}) + O(n)$ ללא שיפור.

האלגוריתם של Karatsuba מיישם טריק מתמטי לצמצום מספר המכפלות ל-3 בלבד: נחשב את שלושת המכפלות הבאות:

$$A = x_1 y_1$$

$$B = x_2 y_2$$

$$C = (x_1 + x_2)(y_1 + y_2)$$

נשים לב כי איבר האמצע מבוטא כעת כ: $C - A - B$.
 $x_1 y_2 + x_2 y_1 = C - A - B$.
 נוסחת הנסיגה לזמן הריצה:

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$

על פי משפט המאסטר (מקרה 1, שכן $\log_2 3 \approx 1.58 \implies n \in o(n^{1.58})$), נקבל סיבוכיות משופרת של:

$$T(n) \in O(n^{\log_2 3}) \approx O(n^{1.58})$$

2. בעיית מציאת שיא דו-ממדי (D Peak Finding)

נתונה מטריצה M בגודל $n \times n$. איבר $M[i][j]$ נקרא שיא מקומי אם הוא גדול או שווה לכל ארבעת שכניו הסמוכים (למעלה, למטה, שמאל, ימין).
 נרצה למצוא שיא כזה בזמן אופטימלי של $O(n)$ (נשים לב שסריקה מלאה של המטריצה דורשת $\Theta(n^2)$ זמן).

אלגוריתם מבוסס מסגרות (MIT 2-D Peak Finding)

- חלק את המטריצה ל-4 תת-מטריצות שוות בגודל $\frac{n}{2} \times \frac{n}{2}$ על ידי סימון של "מסגרת" משותפת המורכבת מהשורה האמצעית והעמודה האמצעית של המטריצה הכוללת (וכן גבולות המטריצה החיצוניים).
- סרוק את כל תאי המסגרת שסומנה (אורכה הכולל הוא $O(n)$) ומצא בהם את האיבר המקסימלי הגלובלי, נסמנו כ- x .
- בחן את שכניו של x במטריצה:
 - אם x גדול או שווה לשכניו, מצאנו שיא גלובלי ומקומי, נחזיר אותו ונסיים.
 - אם קיים שכן y הגדול מ- x , בהכרח y חייב להימצא בתוך אחת מארבע תת-המטריצות הפנימיות (שאינן חלק מהמסגרת).
- נריץ את האלגוריתם באופן רקורסיבי על תת-המטריצה בגודל $\frac{n}{2} \times \frac{n}{2}$ המכילה את השכן y .
 נוסחת הנסיגה לזמן הריצה:

$$T(n) = T\left(\frac{n}{2}\right) + O(n) \in O(n)$$

מובטח כי האלגוריתם יתכנס לשיא מקומי תקין, שכן המעבר לשכן גדול יותר מבטיח עלייה רציפה שאינה יכולה לצאת מגבולות תת-המטריצה שכן גבולותיה חסומים על ידי ערכים קטנים מ- x .

14. מיונים ובעיות בחירה

חסם תחתון למיונים מבוססי השוואה

מיון מבוסס השוואות הוא אלגוריתם המקבל קלט של n איברים ומבצע אינטראקציה עימם אך ורק באמצעות השוואת זוגות $(A[i] > A[j])$.

משפט: חסם תחתון אסימפטוטי למיונים

כל אלגוריתם מיון מבוסס השוואות דורש לפחות $\Omega(n \log n)$ השוואות במקרה הגרוע ביותר ובמקרה הממוצע.

הוכחה:

ניתן לייצג את הרצת האלגוריתם כעץ השוואות בינארי שבו:

- כל קודקוד פנימי מייצג השוואה בודדת $(A[i] \leq A[j])$.
- כל עלה מייצג פלט אפשרי יחיד של פרמוטציה ממוינת של הקלט.
- מכיוון שלמערך בגודל n יש בדיוק $n!$ פרמוטציות אפשריות שונות, עץ השוואות תקין חייב להכיל לפחות $L \geq n!$ עלים כדי לתמוך בכל קלטי המטרה האפשריים.
- גובה עץ בינארי בעל L עלים הוא לפחות $\lceil \log L \rceil$. נסמן את מספר ההשוואות במקרה הגרוע כגובה העץ h :

$$h \geq \log(n!)$$

נשתמש בזהויות לוגריתמיות ובקירוב סטירלינג $(\log(n!) = \Theta(n \log n))$, או באמצעות חסם תחתון פשוט של חצי מהאיברים:

$$\log(n!) = \sum_{i=1}^n \log i \geq \sum_{i=\lceil n/2 \rceil}^n \log i \geq \frac{n}{2} \log \left(\frac{n}{2} \right) = \frac{n}{2} (\log n - 1) \in \Omega(n \log n)$$

מכאן שכל מיון מבוסס השוואות דורש לפחות $\Omega(n \log n)$ השוואות במקרה הגרוע ביותר.

יציבות מיונים (Stable Sorting)

מיון יציב

מיון נקרא יציב אם הוא שומר על הסדר המקורי של איברים בעלי מפתחות זהים כפי שהופיעו במערך הקלט לפני המיון.

- **מיונים יציבים:** מיון מיזוג (Merge-Sort), מיון הכנסה (Insertion-Sort), מיון בועות (Bubble-Sort), מיון מנייה (Counting-Sort).
- **מיונים לא יציבים:** מיון מהיר (Quick-Sort), מיון ערימה (Heap-Sort), מיון בחירה (Selection-Sort).

טכניקה כללית להפיכת כל מיון ליציב

ניתן להמיר כל אלגוריתם מיון לא יציב ליציב על ידי הרחבת מפתח המיון:
 נשנה כל מפתח key להיות זוג סדור $(key, index)$ שבו האיבר השני מייצג את האינדקס המקורי של האיבר במערך הקלט. בעת ביצוע השוואה בין שני איברים:

- אם המפתחות שלהם שונים, נמיין לפיהם.
- אם המפתחות שלהם זהים, נקבע את הסדר לפי האינדקס המקורי שלהם (האינדקס הקטן יותר יקדם).

שינוי זה דורש תוספת זיכרון של $O(n)$ עבור שמירת האינדקסים, אך אינו משנה אסימפטוטית את זמן הריצה של ההשוואות.

מיונים שאינם מבוססי השוואה

1. מיון מנייה (Counting Sort)

מתאים למיון n מספרים שלמים הנמצאים בטווח מוגבל וחסום $[0 \dots R]$. האלגוריתם מקים מערך מונים בגודל $R + 1$, מוחק ומעתיק את האיברים לפי מניית כמות המופעים המדויקת של כל איבר.

- **סיבוכיות זמן:** $O(n + R)$. אם הטווח R הוא ליניארי ביחס למספר האיברים (כלומר $R \in O(n)$), זמן המיון הוא ליניארי אופטימלי $O(n)$.
- **סיבוכיות מקום:** $O(n + R)$.

2. מיון בסיס (Radix Sort)

מתאים למיון n מספרים בגודל d ספרות, כאשר כל ספרה מיוצגת בבסיס b . האלגוריתם מפעיל מיון יציב מובנה (לרוב Counting Sort) על המספרים מהספרה הפחות משמעותית (אחדות) ועד הספרה הכי משמעותית (מאות/אלפים).

⚠ חשיבות היציבות במיון בסיס

תנאי הכרחי להצלחת מיון בסיס הוא שהמיון הפנימי המופעל בכל שלב יהיה יציב. אם המיון הפנימי אינו יציב, הוא עלול לשבש את הסדר הממוין שכבר הושג בסבבים הקודמים עבור הספרות הפחות משמעותיות, והתוצאה הסופית לא תהיה ממוינת.

- **סיבוכיות זמן:** $O(d(n + b))$ (עבור d סבבים של Counting Sort בבסיס b). אם נבחר את הבסיס להיות $b = n$, וטווח המספרים הוא $[0 \dots n^d - 1]$, נוכל למיין את המערך בזמן ליניארי של $O(d \cdot n) = O(n)$ (עבור קבוע d).

בעיית הבחירה (Selection)

בעיית הבחירה עוסקת במציאת האיבר ה- i בגודלו במערך לא ממוין של n איברים. נרצה לפתור אותה בזמן ליניארי $O(n)$ מבלי למיין את המערך כולו.

אלגוריתם Select הליניארי במקרה הגרוע (Median of Medians)

האלגוריתם מיישם בחירה חכמה של איבר ציר (Pivot) המבטיח חלוקה מאוזנת של המערך בכל שלב רקורסיבי:

1. **חלוקה לחמישיות:** נחלק את n איברי המערך לתת-קבוצות בגודל 5 איברים כל אחת (לכל היותר $\lceil \frac{n}{5} \rceil$ חמישיות).
2. **מציאת חציונים:** נמיין כל חמישייה בנפרד בזמן קבוע של $O(1)$, ונשלוף את האיבר האמצעי (החציון) שלהן.
3. **חציון החציונים:** נריץ קריאה רקורסיבית לאלגוריתם Select על קבוצת החציונים שקיבלנו (שגודלה $\lceil \frac{n}{5} \rceil$ כדי למצוא את החציון של החציונים, נסמנו כ- p).
4. **חלוקה סביב הציר:** נבצע חלוקה (Partition) של המערך המקורי סביב הציר p (בדומה ל-Quick-Sort).
5. **המשך רקורסיבי:** נמשיך לסרוק רקורסיבית רק את תת-המערך הרלוונטי המכיל את האינדקס ה- i המבוקש.

פיתוח משוואת הסיבוכיות של חציון החציונים

בזכות בחירת הציר המבוססת על חציון החציונים, מובטח כי לפחות מחצית מחציוני החמישיות קטנים או שווים לציר p . עבור כל חציון כזה, ישנם לפחות 3 איברים בחמישייה שלו הקטנים או שווים לו. לפיכך, מספר האיברים המובטחים להיות קטנים או שווים ל- p הוא לפחות:

$$3 \left(\frac{1}{2} \left\lceil \frac{n}{5} \right\rceil - 2 \right) \geq \frac{3n}{10} - 6$$

באופן סימטרי, ישנם לפחות $\frac{3n}{10} - 6$ איברים הגדולים או שווים ל- p . מכאן שבמקרה הגרוע ביותר, תת-המערך שעליו נריץ את הקריאה הרקורסיה השנייה גודלו לכל היותר $\frac{7n}{10} + 6$ איברים.

נוסחת הנסיגה של זמן הריצה היא:

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$$

נוכיח בשיטת ההצבה כי $T(n) \in O(n)$:

נניח כי $T(n) \leq cn$. נציב במשוואה:

$$T(n) \leq c \frac{n}{5} + c \frac{7n}{10} + an = cn \left(\frac{2}{10} + \frac{7}{10} \right) + an = \frac{9}{10}cn + an$$

עלינו לדרוש:

$$\frac{9}{10}cn + an \leq cn \iff an \leq \frac{1}{10}cn \iff c \geq 10a$$

עבור קבוע c גדול מספיק (לפחות פי 10 מקבוע העבודה הליניארית a), האי-שוויון מתקיים, והסיבוכיות במקרה הגרוע ביותר היא ליניארית אופטימלית:

$$T(n) \in \Theta(n)$$

יישום: מציאת k האיברים הקרובים ביותר לחציון ב- $O(n)$

נתון מערך A בגודל n של מספרים. נרצה להדפיס את k האיברים שערכם קרוב ביותר לערך החציון של המערך בזמן ליניארי $O(n)$ במקרה הגרוע ביותר.

אלגוריתם המימוש

1. נריץ את אלגוריתם Select כדי למצוא את החציון של המערך A , נסמנו כ- m . שלב זה דורש $O(n)$ זמן.
2. נקים מערך עזר חדש B בגודל n , שבו לכל אינדקס נחשב את המרחק המוחלט מהחציון:

$$B[i] = |A[i] - m|$$

שלב זה דורש מעבר ליניארי יחיד של $O(n)$ זמן.

3. נריץ את אלגוריתם Select על מערך המרחקים B כדי למצוא את האיבר ה- k בגודלו ב- B , נסמנו כ- d_k . שלב זה דורש $O(n)$ זמן.

נשים לב שעל פי תכונת אלגוריתם ה-Select, הוא מבטיח כי כל האיברים שמרחקם קטן מ- d_k ימוקמו משמאלו במערך, וכל האיברים שמרחקם גדול מ- d_k ימוקמו מימינו.

4. נסרוק את המערך ונחזיר את כל האיברים $A[i]$ שעבורם מתקיים $|A[i] - m| \leq d_k$. אלו הם בדיוק k האיברים הקרובים ביותר לחציון. סיבוכיות זמן כוללת: $O(n)$ במקרה הגרוע ביותר.

15. תכנון דינמי (Dynamic Programming)

15.1 מבוא והגדרות יסוד

תכנון דינמי הוא פרדיגמה לתכנון אלגוריתמים המיועדת לפתרון בעיות אופטימיזציה מורכבות על ידי פירוקן לתתי-בעיות פשוטות יותר, פתרון תתי-הבעיות פעם אחת בלבד ושמירת פתרונותיהן בזיכרון כדי למנוע חישובים חוזרים.

השיטה פותחה על ידי ריצ'רד בלמן בשנות ה-50 של המאה ה-20. המילה "תכנון" (Programming) בהקשר זה מתייחסת למציאת תוכנית פעולה אופטימלית (במובן המתמטי), ולא לכתיבת קוד במחשב.

כדי שבעיה תתאים לפתרון באמצעות תכנון דינמי, היא חייבת לקיים שני מאפיינים מרכזיים:

תת-מבנה אופטימלי (Optimal Substructure)

בעיה מקיימת תת-מבנה אופטימלי אם ניתן להרכיב פתרון אופטימלי לבעיה מתוך פתרונות אופטימליים לתתי-הבעיות שלה. לדוגמה, המסלול הקצר ביותר מנקודה A לנקודה C העובר דרך B מורכב מהמסלול הקצר ביותר מ-A ל-B ומהמסלול הקצר ביותר מ-B ל-C.

תתי-בעיות חופפות (Overlapping Subproblems)

בעיה מקיימת תתי-בעיות חופפות אם מהלך פתרונה הרקורסיבי הנאיבי גורר חישוב חוזר של תתי-בעיות זהות מספר רב של פעמים (בדרך כלל אקספוננציאלי). במקום לחשב מחדש, שומרים את התוצאות במבנה נתונים.

אינטואיציה: מניעת חישובים כפולים

תכנון דינמי הוא למעשה מסחר מושכל במשאבים: אנו משתמשים בתוספת קטנה של זיכרון (Space Complexity) על מנת להשיג שיפור עצום (לעתים קרובות מאקספוננציאלי לפולינומי) בזמן הריצה (Time Complexity). אנו פותרים כל תת-בעיה פעם אחת בלבד, רושמים את התשובה ב"פנקס" (טבלה או מערך), ובפעם הבאה שאנו נתקלים בה אנו פשוט קוראים את התשובה מהפנקס בזמן קבוע של $O(1)$.

מתודולוגיית ארבעת השלבים לפיתוח אלגוריתם DP:

1. **אפיון המבנה של פתרון אופטימלי:** הבנת הקשר בין פתרון הבעיה לפתרונות תתי-הבעיות.
2. **הגדרת פונקציית תת-הבעיה ונוסחת נסיגה:** הגדרת הערך האופטימלי בצורה רקורסיבית ומקרי בסיס.
3. **חישוב הערך האופטימלי:** מילוי הטבלה מלמטה למעלה (Tabulation) או מלמעלה למטה עם זיכרון (Memoization).
4. **שחזור הפתרון האופטימלי מהטבלה:** מעקב לאחור (Traceback) למציאת הבחירות שבוצעו בפועל.

אסטרטגיית פתרון: שתי גישות המימוש

- **Top-Down (Memoization) עם תזכור:** כתיבת פתרון רקורסיבי נאיבי, ומעבר עליו עם מערך או מילון. גישה זו יעילה כאשר עזר. בכל תחילת פונקציה בודקים אם הערך כבר חושב; אם כן, מחזירים אותו מייד. גישה זו יעילה כאשר אין צורך לחשב את כל תתי-הבעיות האפשריות.

- **Bottom-Up עם חישוב טבלאי (Tabulation):** מילוי המערך או הטבלה בלולאות מהאיברים הקטנים (מקרי הבסיס) ועד לאיבר המבוקש. גישה זו מונעת את תקורת קריאות הרקורסיה ומאפשרת לעיתים קרובות אופטימיזציית מקום על ידי מחיקת שורות קודמות שאינן נחוצות (Stack Overflow) עוד.

15.2 מספרי פיבונאצ'י (Fibonacci)

סדרת פיבונאצ'י מוגדרת על ידי תנאי ההתחלה הבאים ונוסחת נסיגה:

משפט: נוסחת הנסיגה של פיבונאצ'י

$$F(0) = 0$$

$$F(1) = 1$$

לכל $n \geq 2$:

$$F(n) = F(n-1) + F(n-2)$$

הגישה הרקורסיבית הנאיבית:

קוד המחשב את הנוסחה ישירות על ידי קריאות רקורסיביות כפולות.

```
def fib_naive(n):
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)
```

⚠️ זהירות - טעות נפוצה: סיבוכיות הגישה הרקורסיבית הנאיבית

זמן הריצה של האלגוריתם הרקורסיבי הנאיבי הוא מעריכי:

$$T(n) = T(n-1) + T(n-2) + O(1) \implies T(n) \in \Theta(\phi^n)$$

כאשר $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ הוא יחס הזהב. הסיבה לכך היא ריבוי חישובים חוזרים. לדוגמה, כדי לחשב את $F(5)$, אנו מחשבים את $F(3)$ פעמיים, את $F(2)$ שלוש פעמים, ואת $F(1)$ חמש פעמים.

פתרון בתכנון דינמי (Bottom-Up Tabulation):

נמלא מערך בגודל $n+1$ משמאל לימין:

```
def fib_dp(n):
    if n <= 1:
        return n
    F = [0] * (n + 1)
```

```
F[1] = 1
for i in range(2, n + 1):
    F[i] = F[i - 1] + F[i - 2]
return F[n]
```

סיבוכיות זמן: $O(n)$ - לולאה פשוטה. סיבוכיות מקום: $O(n)$ לשמירת המערך.

🔗 הערת המרצה (מעבר מתמטי): אופטימיזציית מקום בפיבונצ'י

שים לב כי לצורך חישוב $F(i)$ אנו נדרשים אך ורק לשני הערכים הקודמים $F(i-1)$ ו- $F(i-2)$. אין שום צורך לשמור את כל המערך בזיכרון. נוכל להשתמש בשני משתנים בלבד המעודכנים בכל איטרציה, ובכך לצמצם את סיבוכיות המקום ל- $O(1)$ בלבד!

```
:def fib_space_optimized(n)
    :if n <= 1
        return n
    prev2, prev1 = 0, 1
    :for i in range(2, n + 1)
        curr = prev1 + prev2
        prev2 = prev1
        prev1 = curr
    return prev1
```

15.3 תת-סדרה הבלתי תלויה המקסימלית במערך (MIS in Path)

נתון מערך של מספרים $X = x_1, x_2, \dots, x_n$ שבהם לכל איבר יש משקל $w_i \geq 0$. עלינו למצוא תת-סדרה של איברים שאינם סמוכים זה לזה במערך (ללא אינדקסים עוקבים i ו- $i+1$), אשר סכום משקליהם הוא מקסימלי.

הגדרת תת-הבעיה $f(i)$:

הסכום המקסימלי שניתן לקבל מתוך תת-המערך המכיל את האיברים $\{x_1, \dots, x_i\}$.

📌 משפט: נוסחת הנסיגה ל-MIS במערך

מקרי בסיס:

$$f(0) = 0$$

$$f(1) = w_1$$

לכל $2 \leq i \leq n$:

$$f(i) = \max\{f(i-1), f(i-2) + w_i\}$$

הסבר רעיוני:

עבור האיבר i , יש לנו שתי החלטות אפשריות:

1. **לא לקחת את x_i** : הרווח הוא הפתרון האופטימלי ללא האיבר i , כלומר $f(i-1)$.
2. **כן לקחת את x_i** : אנו מקבלים מיידית את המשקל w_i . מאחר שאסור לקחת איברים סמוכים, אנו מחויבים לא לקחת את x_{i-1} , ולכן שאר האיברים נבחרים באופן אופטימלי מתוך $i-2$ האיברים הראשונים, מה שמניב רווח של $f(i-2) + w_i$.

אינטואיציה: הדילמה של כל איבר

בכל צעד של תכנון דינמי אנו מדמיינים שאנו עומדים מול האיבר האחרון ברישא. הדילמה שלנו היא פשוטה: "האם לכלול אותו או לדלג עליו?". אם אנו כוללים אותו, אנו משלמים מחיר מסוים (במקרה זה, איסור בחירה של השכן שלו) ומרוויחים את ערכו. אם אנו מדלגים עליו, אנו חופשיים לבחור את השכן.

פסאודו-קוד ומילוי מערך:

```
def mis_path_dp(W):
    n = len(W)
    if n == 0: return 0
    if n == 1: return W[0]
    F = [0] * n
    F[0] = W[0]
    F[1] = max(W[0], W[1])
    for i in range(2, n):
        F[i] = max(F[i-1], F[i-2] + W[i])
    return F
```

סיבוכיות זמן: $O(n)$, סיבוכיות מקום: $O(n)$ (ניתן לצמצום ל- $O(1)$ באמצעות שני משתנים).

שחזור הפתרון האופטימלי:

נרוץ מהסוף להתחלה על גבי מערך ה-DP שחושב F :

```
def reconstruct_mis_path(F, W):
    i = len(F) - 1
    chosen = []
    while i >= 0:
        if i == 0:
            chosen.append(0)
            break
        elif i == 1:
            if W[1] > W[0]:
                chosen.append(1)
            else:
                chosen.append(0)
            break
        # For i > 1, we check if including W[i] is better
        if F[i] > F[i-1]:
            chosen.append(i)
            i -= 2
        else:
            i -= 1
```

```

# אם הערך הנוכחי שווה לערך ללא האיבר, דילגנו עליו
if F[i] == F[i - 1]:
    i -= 1
else:
    chosen.append(i)
    i -= 2
return sorted(chosen)

```

⚠ מלכודת נפוצה! מעקב אחר אינדקסים בשחזור

בעת שחזור פתרון, יש להיזהר מגישה לאינדקסים שליליים במערך. כאשר אנו מבצעים השוואה כמו $F[i] = F[i - 1]$, עלינו לוודא ש- $i - 1 \geq 0$. לכן במקרי הקצה של $i = 0$ ו- $i = 1$ יש לטפל בנפרד מחוץ ללולאה הראשית כפי שמוצג בקוד לעיל.

15.4 תזמון מקטעי ממושקלים (Weighted Interval Scheduling)

נתונה קבוצה של n משימות (intervals). לכל משימה j יש זמן התחלה s_j , זמן סיום f_j ומשקל (רווח) $v_j > 0$. שתי משימות הן חופפות אם הן משתפות קטע זמן כלשהו. עלינו למצוא קבוצת משימות חוקית (שאין בה שתי משימות חופפות) אשר ממקסמת את סך הרווח הכולל.

עיבוד מקדים של המשימות:

נמיין את המשימות בסדר עולה של זמני הסיום שלהן:

$$f_1 \leq f_2 \leq \dots \leq f_n$$

פונקציית העזר $p(j)$

לכל משימה j , נגדיר את $p(j)$ להיות האינדקס של המשימה האחרונה $i < j$ שאינה חופפת למשימה j (כלומר המשימה שמסתיימת לפני ש- j מתחילה: $f_i \leq s_j$). אם אין משימה כזו, נגדיר $p(j) = 0$.

משפט: נוסחת הנסיגה לבעיית תזמון מקטעי ממושקלים

נגדיר את $M(j)$ להיות הרווח האופטימלי שניתן להפיק מתוך המשימות $\{1, \dots, j\}$. בסיס:

$$M(0) = 0$$

לכל $1 \leq j \leq n$:

$$M(j) = \max\{M(j-1), v_j + M(p(j))\}$$

הסבר רעיוני:

עבור המשימה ה- j , אנו שוקלים שתי אפשרויות:

1. לא לכלול את המשימה j : הרווח הוא הפתרון האופטימלי עבור $j - 1$ המשימות הראשונות, כלומר $M(j - 1)$.
2. כן לכלול את המשימה j : אנו מקבלים את הרווח v_j . המשימה הקודמת הכי קרובה שיכולה להשתלב איתה היא $p(j)$, ולכן אנו מוסיפים את הרווח האופטימלי של המשימות עד $p(j)$, כלומר $M(p(j))$.

🔗 טיפ למבחן: חישוב יעיל של $p(j)$ באמצעות חיפוש בינארי

חישוב נאיבי של $p(j)$ לכל המשימות דורש מעבר על כל המשימות הקודמות ולוקח $O(n^2)$ זמן. מכיוון שהמשימות ממוינות לפי זמני סיום, ניתן למצוא את המשימה ה- i האחרונה המקיימת $f_i \leq s_j$ באמצעות חיפוש בינארי על זמני הסיום. הדבר מצמצם את חישוב כל ערכי $p(j)$ לזמן כולל של $O(n \log n)$ בלבד!

פסאודו-קוד לאלגוריתם:

```
def compute_p(intervals):
    # ממוינים לפי זמני סיום f
    n = len(intervals)
    p = [0] * (n + 1)
    for j in range(1, n + 1):
        # intervals[i-1].f <= intervals[j-1].s
        low, high = 1, j - 1
        target = intervals[j - 1][0] # s_j
        ans = 0
        while low <= high:
            mid = (low + high) // 2
            if intervals[mid - 1][1] <= target: # f_mid <= s_j
                ans = mid
                low = mid + 1
            else:
                high = mid - 1
        p[j] = ans
    return p

def wis_dp(intervals, V):
    n = len(intervals)
    p = compute_p(intervals)
    M = [0] * (n + 1)
    for j in range(1, n + 1):
        M[j] = max(M[j - 1], V[j - 1] + M[p[j]])
    return M, p
```

סיבוכיות זמן כוללת: $O(n \log n)$ בשל המיון והחיפוש הבינארי. סיבוכיות מקום: $O(n)$ לשמירת הטבלאות.

שחזור המשימות שנבחרו:

```
def reconstruct_wis(M, p, intervals, j):
    if j == 0:
        return []
    if M[j] > M[j - 1]:
        return reconstruct_wis(M, p, intervals, p[j]) + [j]
    else:
        return reconstruct_wis(M, p, intervals, j - 1)
```

15.5 תת-סדרה משותפת ארוכה ביותר (LCS)

נתונות שתי סדרות: $X = x_1, x_2, \dots, x_m$ ו- $Y = y_1, y_2, \dots, y_n$. עלינו למצוא את אורך תת-הסדרה המשותפת הארוכה ביותר של X ו- Y (תת-סדרה של איברים המופיעים בשתי הסדרות באותו סדר מקורי, אך לא בהכרח ברצף).

הגדרת תת-הבעיה $c(i, j)$:

אורך תת-הסדרה המשותפת הארוכה ביותר של הרישות $X_i = x_1, \dots, x_i$ ו- $Y_j = y_1, \dots, y_j$.

משפט: נוסחת הנסיגה של LCS

בסיס:

$$c(i, 0) = 0$$

$$c(0, j) = 0$$

לכל $1 \leq i \leq m$ ולכל $1 \leq j \leq n$:

$$c(i, j) = \begin{cases} c(i-1, j-1) + 1 & x_i = y_j \\ \max\{c(i-1, j), c(i, j-1)\} & x_i \neq y_j \end{cases}$$

אינטואיציה: השוואה מקצה לקצה

אנו מתבוננים באיברים האחרונים של שתי הרישות, x_i ו- y_j :

- אם הם זהים, הם בהכרח מהווים חלק מתת-הסדרה המשותפת האופטימלית של רישות אלו. לכן אנו סופרים התאמה אחת (+1) וממשיכים לפתור את הבעיה עבור הרישות ללא איברים אלו ($c(i-1, j-1)$).
- אם הם שונים, לפחות אחד מהם אינו יכול להשתתף ב-LCS. לכן אנו בוחנים את שתי האפשרויות (השמטת x_i או השמטת y_j) ולוקחים את המקסימום מביניהן.

מילוי טבלה דו-ממדית C :

```
def lcs_dp(X, Y):
    m, n = len(X), len(Y)
```

```

C = [[0] * (n + 1) for _ in range(m + 1)]
for i in range(1, m + 1):
    for j in range(1, n + 1):
        if X[i - 1] == Y[j - 1]:
            C[i][j] = C[i - 1][j - 1] + 1
        else:
            C[i][j] = max(C[i - 1][j], C[i][j - 1])
return C

```

סיבוכיות זמן: $O(m \cdot n)$ - מעבר על כל תאי הטבלה. סיבוכיות מקום: $O(m \cdot n)$ לשמירת הטבלה.

שחזור סדרת ה-LCS בפועל:

נבצע סריקה לאחור מהפינה הימנית התחתונה $C[m][n]$:

```

def reconstruct_lcs(C, X, Y):
    i, j = len(X), len(Y)
    lcs = []
    while i > 0 and j > 0:
        if X[i - 1] == Y[j - 1]:
            lcs.append(X[i - 1])
            i -= 1
            j -= 1
        elif C[i - 1][j] >= C[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return "".join(reversed(lcs))

```

⚠️ זהירות - טעות נפוצה: שחזור ה-LCS בטבלה

במהלך שחזור לאחור, אסור לצרף איבר ל-LCS סתם כך על בסיס השוואת ערכי תאים. אנו מצרפים איבר אך ורק כאשר מתקיימת התאמה ישירה בין התווים של שתי הסדרות, כלומר $X[i - 1] == Y[j - 1]$, ואז אנו נסוגים באלכסון לתא $C[i - 1][j - 1]$. בכל מקרה אחר אנו פשוט נסוגים שמאלה או למעלה לפי התא בעל הערך הגדול יותר.

15.6 כפל שרשרת מטריצות (Matrix Chain Multiplication - MCM)

נתונה שרשרת של n מטריצות $A_1, A_2, \dots, A_n$. אנו מעוניינים לחשב את המכפלה $A_1 A_2 \dots A_n$. פעולת כפל מטריצות היא אסוציאטיבית (ניתן להציב סוגריים בכל סדר), אך מספר פעולות הכפל הסקלריות הנדרש משתנה דרמטית בהתאם לסדר הסוגריים.

לדוגמה, כפל מטריצה $P \times Q$ במטריצה $Q \times R$ דורש $P \cdot Q \cdot R$ כפלים סקלאריים. עלינו למצוא סדר הצבת סוגריים הממזער את מספר כפלי הסקלרים הכולל.

- קלט הממדים: מערך ממדים $p = (p_0, p_1, \dots, p_n)$, כך שהמטריצה A_i היא בגודל $p_{i-1} \times p_i$.

בעיית כפל שרשרת מטריצות

מספר סדרי הסוגריים האפשריים עבור שרשרת באורך n מיוצג על ידי מספרי קטלן (Catalan numbers):

$$C_{n-1} = \frac{1}{n} \binom{2n-2}{n-1} \in \Omega\left(\frac{4^n}{n^{1.5}}\right)$$

קצב גידול זה הוא אקספוננציאלי מהיר, ולכן לא ניתן לבצע חיפוש נאיבי על כל האפשרויות.

הגדרת תת-הבעיה $m(i, j)$:

מספר הכפלים הסקלריים המינימלי הנדרש לחישוב המכפלה של שרשרת המטריצות $A_i \dots A_j$ (עבור $1 \leq i \leq j \leq n$).

משפט: נוסחת הנסיגה ל-MCM

בסיס (מטריצה בודדת אינה דורשת כפל):

$$m(i, i) = 0$$

לכל $1 \leq i < j \leq n$:

$$m(i, j) = \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\}$$

הסבר רעיוני:

הכפל האחרון בשרשרת המטריצות $A_i \dots A_j$ יתבצע בין שתי מטריצות תוצאה זמניות: המכפלה המשותפת של החצי השמאלי $A_i \dots A_k$, והמכפלה של החצי הימני $A_{k+1} \dots A_j$, עבור אינדקס חלוקה כלשהו k .

- עלות חישוב החלק השמאלי היא $m(i, k)$.
- עלות חישוב החלק הימני היא $m(k+1, j)$.
- עלות הכפלת שתי המטריצות שנוצרו (שהן בגודל $p_{i-1} \times p_k$ ו- $p_k \times p_j$) היא $p_{i-1}p_kp_j$.
אנו סורקים את כל נקודות החיתוך האפשריות k בטווח שבין i ל- $j-1$ ובחרים במינימלית.

הערת המרצה (מעבר מתמטי): סדר מילוי אלכסוני

שים לב לאופן מילוי הטבלה $m[i, j]$: חישוב הערך בתא i, j מסתמך על תאים הנמצאים משמאלו ($m(i, k)$) ומתחתיו ($m(k+1, j)$) בטבלה. לכן, לא ניתן למלא את הטבלה שורה אחר שורה או עמודה אחר עמודה בצורה פשוטה. עלינו למלא את הטבלה לפי אלכסונים, כלומר לפי אורך השרשרת $l = j - i + 1$ ההולך וגדל מ-2 ועד n .

פסאודו-קוד לאלגוריתם:

```
def matrix_chain_order(p):
    n = len(p) - 1
```

```

m = [[0] * (n + 1) for _ in range(n + 1)]
# s האופטימלי שבו התבצע הפיצול k-תשומר את אינדקס ה
s = [[0] * (n + 1) for _ in range(n + 1)]

# מייצג את אורך השרשרת l
for l in range(2, n + 1):
    for i in range(1, n - l + 2):
        j = i + l - 1
        m[i][j] = float('inf')
        for k in range(i, j):
            q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j]
            if q < m[i][j]:
                m[i][j] = q
                s[i][j] = k

return m, s

```

סיבוכיות זמן: $O(n^3)$ - שלוש לולאות מקוננות (אורך שרשרת, שורת התחלה, נקודת פיצול k). סיבוכיות מקום: $O(n^2)$ לשמירת הטבלאות.

שחזור והדפסת סדר הסוגריים האופטימלי:

נפעיל פונקציה רקורסיבית העושה שימוש בטבלה s :

```

def print_optimal_parens(s, i, j):
    if i == j:
        return f"A{i}"
    else:
        k = s[i][j]
        left = print_optimal_parens(s, i, k)
        right = print_optimal_parens(s, k + 1, j)
        return f"({left} * {right})"

```

15.7 בעיית תרמיל הגב בשלמים (Integer Knapsack)

נתונים n חפצים. לכל חפץ i יש משקל שלם $w_i > 0$ ושווי $v_i > 0$. ברשותנו תרמיל בעל קיבולת משקל מקסימלית B (שלם). עלינו לבחור קבוצת חפצים $I \subseteq \{1, \dots, n\}$ שמשקלה הכולל אינו עולה על B ושוויה הכולל הוא מקסימלי. זוהי בעיית תרמיל הגב 0-1, שבה לא ניתן לחלק חפצים.

הגדרת תת-הבעיה $f(j, b)$:

השווי המקסימלי שניתן להשיג מתוך תת-קבוצת החפצים $\{1, \dots, j\}$ עם קיבולת תרמיל מוגבלת ל- b (עבור $0 \leq b \leq B$ ו- $0 \leq j \leq n$).

משפט: נוסחת הנסיגה לבעיית תרמיל הגב

בסיס (כשאינן חפצים או שקיבולת התיק היא 0):

$$f(0, b) = 0$$

$$f(j, 0) = 0$$

לכל $1 \leq j \leq n$ ולכל $0 \leq b \leq B$:

$$f(j, b) = \begin{cases} f(j-1, b) & w_j > b \\ \max\{f(j-1, b), v_j + f(j-1, b - w_j)\} & w_j \leq b \end{cases}$$

הסבר רעיוני:

עבור החפץ ה- j בקיבולת הנוכחית b :

- אם משקלו גדול מ- b , לא ניתן להכניסו לתיק, והפתרון שווה ל- $f(j-1, b)$.
- אם משקלו קטן או שווה ל- b , נבחר במקסימום מבין:
 1. אי-הכללת החפץ j : הרווח הוא $f(j-1, b)$.
 2. הכללת החפץ j : הרווח הוא שווי החפץ v_j ועוד השווי האופטימלי שניתן לקבל מ- $j-1$ החפצים הראשונים עם קיבולת שנותרה של $b - w_j$, כלומר $f(j-1, b - w_j)$.

⚠ מלכודת נפוצה: סיבוכיות פסאודו-פולינומית!

זמן הריצה של מילוי הטבלה הוא $O(n \cdot B)$. לכאורה מדובר בסיבוכיות פולינומית, אך למעשה היא פסאודו-פולינומית.

גודל הקלט הנדרש לייצוג המספר B בזיכרון הוא לוגריתמי, כלומר $\log B$ ביטים. זמן הריצה גדל באופן ליניארי ביחס ל- B עצמו, ולכן הוא אקספוננציאלי בגודל ייצוג הקלט ($2^{\log B}$). אם B יהיה עצום (למשל $B = 2^{64}$), זמן הריצה יהיה בלתי אפשרי, למרות שגודל הקלט קטן מאוד.

מילוי הטבלה הדו-ממדית A :

```
def knapsack_01_dp(V, W, B):
    n = len(V)
    A = [[0] * (B + 1) for _ in range(n + 1)]
    for j in range(1, n + 1):
        for b in range(0, B + 1):
            if W[j - 1] > b:
                A[j][b] = A[j - 1][b]
            else:
                A[j][b] = max(A[j - 1][b], V[j - 1] + A[j - 1][b - W[j - 1]])
    return A
```

סיבוכיות זמן: $O(n \cdot B)$, סיבוכיות מקום: $O(n \cdot B)$. אם מעוניינים רק בערך המקסימלי ללא שחזור, ניתן לשמור רק שתי שורות בזיכרון ולצמצם את המקום ל- $O(B)$.

שחזור החפצים שנבחרו:

נרוץ לאחור מהפינה הימנית התחתונה $A[n][B]$:

```
def reconstruct_knapsack(A, W, B):
    j, b = len(W), B
    chosen = []
    while j > 0:
        if A[j][b] != A[j - 1][b]:
            chosen.append(j - 1) # אינדקס החפץ (0-based)
            b -= W[j - 1]
        j -= 1
    return sorted(chosen)
```

15.8 קבוצה בלתי תלויה מקסימלית בעץ (MIS in a Tree)

נתון עץ מושרש $T = (V, E)$ בעל n קודקודים. לכל קודקוד v יש משקל (מספר מוזמנים) $f_v \geq 0$. עלינו למצוא קבוצת קודקודים עצמאית (בלתי תלויה) מקסימלית $I \subseteq V$ כך שאין בקבוצה שני קודקודים שיש ביניהם קשת (כלומר יחסי אב ובן בעץ).

הגדרת פונקציות העזר:

בשל המבנה ההיררכי של עץ, נגדיר לכל קודקוד v שתי פונקציות DP:

- $I(v)$: מספר המוזמנים האופטימלי מתת-העץ ששורשו v , בהינתן שמזמינים את v .
- $E(v)$: מספר המוזמנים האופטימלי מתת-העץ ששורשו v , בהינתן שלא מזמינים את v .

משפט: נוסחאות הנסיגה של MIS בעץ

מקרה בסיס (עבור עלה v):

$$I(v) = f_v$$

$$E(v) = 0$$

לכל קודקוד פנימי v :

$$I(v) = f_v + \sum_{u \in \text{Children}(v)} E(u)$$

$$E(v) = \sum_{u \in \text{Children}(v)} \max\{I(u), E(u)\}$$

הסבר רעיוני:

- אם בחרנו להזמין את v ($I(v)$): אסור להזמין אף אחד מבניו הישירים u . לכן אנו מחויבים להשתמש בפתרון של אי-הזמנת הבנים, כלומר $E(u)$.
- אם בחרנו לא להזמין את v ($E(v)$): אין הגבלה על הבנים u . לכל בן נוכל לבחור את הטוב ביותר מבין שתי האפשרויות: להזמין או לא להזמין, כלומר $\max\{I(u), E(u)\}$.

🔗 הערת המרצה (מעבר מתמטי): סדר חישוב באמצעות Post-Order

כדי לחשב את הערכים עבור קודקוד v , אנו חייבים שכל בנוי u יחושבו לפניו. סדר הסריקה המתאים לכך בעץ הוא **Post-Order (סריקה מאוחרת)**. האלגוריתם מבצע סריקה יחידה של העץ, מה שמקנה לו זמן ריצה ליניארי מהיר של $O(n)$.

מימוש האלגוריתם:

```
class TreeNode:
    def __init__(self, f):
        self.f = f
        self.children = []
        self.I = 0
        self.E = 0

def solve_mis_tree(root):
    def post_order_dp(v):
        if v is None:
            return

        for u in v.children:
            post_order_dp(u)

        v.I = v.f
        v.E = 0
        for u in v.children:
            v.I += u.E
            v.E += max(u.I, u.E)

    post_order_dp(root)
    return max(root.I, root.E)
```

סיבוכיות זמן: $O(n)$ - סריקה יחידה על פני n קודקודים. סיבוכיות מקום: $O(n)$ עבור מחסנית הרקורסיה ושמירת הערכים בקודקודים.

15.9 תת-סדרה עולה ארוכה ביותר (LIS)

נתונה סדרת מספרים $X = x_1, x_2, \dots, x_n$. עלינו למצוא את אורך תת-הסדרה הארוכה ביותר של X שהיא מונוטונית עולה (לא בהכרח רציפה).

⚠️ זהירות - טעות נפוצה: תלות של LIS באיבר האחרון

אם נגדיר את $f(i)$ להיות סתם אורך ה-LIS ברישא $x_1 \dots x_i$, לא נוכל לבנות נוסחת נסיגה תקינה. הסיבה היא שכדי להחליט אם לצרף את x_i לתת-סדרה קודמת, אנו חייבים לדעת מהו האיבר האחרון באותה סדרה (כדי לוודא ש- x_i גדול ממנו).

פתרון הבעיה דורש שינוי של הגדרת הפונקציה: נגדיר את $f(i)$ להיות אורך ה-LIS ברישא המסתיימת בהכרח באיבר x_i (כלומר x_i הוא האיבר האחרון בה).

משפט: נוסחת הנוסחה של LIS

לכל $1 \leq i \leq n$:

$$f(i) = 1 + \max \left(\{f(j) \mid 1 \leq j < i \text{ and } x_j \leq x_i\} \cup \{0\} \right)$$

הפתרון לבעיה המקורית יהיה המקסימום של ערכי ה-DP על פני כל אינדקסי הסיום האפשריים:

$$\text{LIS_Length} = \max_{1 \leq i \leq n} f(i)$$

הסבר רעיוני:

האיבר x_i משתתף בסדרה והוא האחרון בה. האיבר הקודם לו בתת-הסדרה חייב להיות איבר x_j כלשהו משמאלו ($j < i$) המקיים $x_j \leq x_i$. אנו סורקים את כל האינדקסים הקודמים הללו, מוצאים את זה בעל אורך ה-LIS המקסימלי $f(j)$, ומוסיפים 1 עבור האיבר x_i עצמו.

מימוש האלגוריתם:

```
def lis_dp(X):
    n = len(X)
    if n == 0: return 0
    F = [1] * n # בסיס: כל איבר לבדו מהווה סדרה באורך 1

    for i in range(1, n):
        for j in range(0, i):
            if X[j] <= X[i]:
                F[i] = max(F[i], F[j] + 1)
    return max(F)
```

סיבוכיות זמן: $O(n^2)$ בשל שתי הלולאות המקוננות. סיבוכיות מקום: $O(n)$ לשמירת המערך F (לא ניתן לצמצום מאחר וגישה לכל תא קודם אפשרית).

טיפ למבחן: שיחזור ה-LIS בפועל

כדי לשחזר את תת-הסדרה עצמה, נחזיק מערך נוסף P בגודל n , שבו לכל אינדקס i נשמור את האינדקס של האיבר הקודם j שבאמצעותו השגנו את המקסימום בחישוב של $F[i]$. לאחר סיום מילוי המערכים, נתחיל מאינדקס המקסימום הכללי במערך F , וננווט לאחור באמצעות המצביעים במערך P .

15.10 בעיית המזכירות / חלוקת סטודנטים לקבוצות

אנו נדרשים לחלק רשימה ממוינת אלפביתית של n סטודנטים ל- k קבוצות לימוד רצופות, כך שסכום החברויות הפנימיות בתוך הקבוצות יהיה מקסימלי.

- **מגבלה:** בכל קבוצה חייבים להיות לפחות 2 סטודנטים.
- **קלט חברויות:** מטריצה בינארית סימטרית A בגודל $n \times n$.
- **ערך עזר $R(x, y)$:** מספר החברויות בתוך הקטע שבין הסטודנט x לסטודנט y .

הגדרת תת-הבעיה $f(i, j)$:

מספר החברויות המקסימלי שניתן לכבד בחלוקת i הסטודנטים הראשונים ברשימה ל- j קבוצות לימוד חוקיות (עבור $1 \leq i \leq n$ ו- $1 \leq j \leq k$).

משפט: נוסחת הנסיגה של בעיית המזכירות

לכל i, j :

$$f(i, j) = \max_{2 \leq s \leq i-2(j-1)} \{f(i-s, j-1) + R(i-s+1, i)\}$$

כאשר s מייצג את גודל הקבוצה האחרונה (הקבוצה ה- j). הגבול התחתון של s הוא 2 (לפחות 2 סטודנטים בקבוצה), והגבול העליון הוא $i - 2(j - 1)$ כדי להבטיח שישארו מספיק סטודנטים לחלוקה חוקית של $j - 1$ הקבוצות הקודמות.

הערת המרצה (מעבר מתמטי): עיבוד מקדים $R(x, y)$ ב- $O(n^2)$

אם נחשב את ערכי $R(x, y)$ באופן נאיבי בכל תא, סיבוכיות האלגוריתם הכללית תעלה ל- $O(n^4 k)$. כדי לייעל זאת, נבצע עיבוד מקדים דו-שלבי בזמן $O(n^2)$ בלבד:

1. **חישוב מטריצה B מלמטה למעלה:** $B[i, j]$ הוא מספר חבריו של j בקבוצה $\{i, \dots, j\}$.

$$B[i, j] = A[i, j] + B[i + 1, j] \quad (\text{with } B[j, j] = 0)$$

2. **חישוב מטריצה R שורה-שורה:** $R[i, j]$ הוא סך כל החברויות הפנימיות בתוך הקבוצה $\{i, \dots, j\}$.

$$R[i, j] = B[i, j] + R[i, j - 1] \quad (\text{with } R[i, i] = 0)$$

לאחר מכן, הגישה לכל ערך $R(x, y)$ ב-DP המרכזי תתבצע ב- $O(1)$.

```
def student_grouping_dp(A, n, k):
    # 1. עיבוד מקדים למטריצת B
    B = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        for j in range(i + 1, n):
            B[i][j] = A[i][j] + B[i + 1][j]

    # 2. עיבוד מקדים למטריצת R
    R = [[0] * n for _ in range(n)]
    for i in range(n):
```

```

for j in range(i + 1, n):
    R[i][j] = B[i][j] + R[i][j - 1]

# 3. מרבזית DP מילורי טבלת
# f[i][j] של מייצג חלוקה של i סטודנטים (1-based) קבוצות j-ל (1-based)
f = [[0] * (k + 1) for _ in range(n + 1)]

for j in range(1, k + 1):
    for i in range(2 * j, n + 1):
        if j == 1:
            f[i][j] = R[0][i - 1]
        else:
            max_val = -1
            for s in range(2, i - 2 * (j - 1) + 1):
                val = f[i - s][j - 1] + R[i - s][i - 1]
                if val > max_val:
                    max_val = val
            f[i][j] = max_val

return f[n][k]

```

סיבוכיות זמן כוללת: $O(n^2 k)$ - שיפור עצום לעומת $O(n^4 k)$ הנאיבי. סיבוכיות מקום: $O(n^2 + nk)$ לשמירת הטבלאות.

15.11 משחק השטרות על לוח

בשורה רציפה מונחים n שטרות כסף בעלי ערכים $v_1, v_2, \dots, v_n \geq 0$. שני שחקנים משחקים בתורות. בכל תור, השחקן רשאי לקחת שטר בודד מאחד משני קצוות השורה (השמאלי ביותר או הימני ביותר). המשחק מסתיים כשאינ שטרות. עלינו לחשב את סכום הכסף המקסימלי שהשחקן הראשון יכול להבטיח לעצמו (בהנחה ששני השחקנים משחקים בצורה מושלמת).

הגדרת תת-הבעיה $p(i, j)$:

הסכום המקסימלי שהשחקן שזהו תורו כעת יכול להבטיח לעצמו מתוך תת-סדרת השטרות שנותרה על הלוח: $v_i, \dots, v_j$ (עבור $1 \leq i \leq j \leq n$).

משפט: נוסחת הנסיגה של משחק השטרות

בסיס (כשנותר שטר בודד):

$$p(i, i) = v_i$$

לכל $1 \leq i < j \leq n$:

$$p(i, j) = \left(\sum_{k=i}^j v_k \right) - \min \{ p(i+1, j), p(i, j-1) \}$$

(נשים לב: בגלל המגבלה על Hebrew ב-LaTeX, המילה "hidden" או ביטוי מחוץ למתמטיקה מתאים כאן. הנוסחה בתוך ה-LaTeX נקיייה מעברית).

הסבר רעיוני:

זהו משחק סכום אפס. סכום השטרות הכולל בקטע הוא $\sum_{k=i}^j v_k$.

- אם ניקח את השטר השמאלי v_i , ליריב יישאר הקטע $v_{i+1} \dots v_j$, שמתוכו הוא יפיק אופטימלית $p(i+1, j)$. לכן אנו נקבל את כל יתרת הסכום של הקטע: $\sum_{k=i}^j v_k - p(i+1, j)$.
 - אם ניקח את השטר הימני v_j , בניתוח זהה אנו נקבל מהקטע שנותר שווי של: $\sum_{k=i}^j v_k - p(i, j-1)$.
- אנו שואפים למקסם את הרווח שלנו, ולכן אנו בוחרים באפשרות שבה היריב מקבל את המינימום מבין שתי האפשרויות שלו (מה שממקסם את היתרה שנותרת לנו).

אינטואיציה: היריב משחק מושלם

תכנון דינמי במשחקים דו-שחקנים (כמו מינימקס) מניח תמיד שהשחקן השני מנסה להזיק לנו ולשחק בצורה הטובה ביותר עבורו. לכן אנו לא בונים על "טעויות" של השחקן השני, אלא בוחרים בצעד שמבטיח לנו את התוצאה הטובה ביותר גם מול מענה מושלם שלו.

הערת המרצה (מעבר מתמטי): שימוש בסכומי רישות (Prefix Sums)

חישוב ישיר של סכום הטווח $\sum_{k=i}^j v_k$ לוקח $O(n)$ זמן, ומעלה את סיבוכיות ה-DP הכוללת ל- $O(n^3)$. כדי לייעל זאת, נבצע עיבוד מקדים בזמן $O(n)$ ונבנה מערך סכומי רישות S בגודל $n+1$, שבו $S[t]$ הוא סכום t האיברים הראשונים. סכום הטווח יחושב בזמן קבוע של $O(1)$ לפי:

$$\sum_{k=i}^j v_k = S[j] - S[i-1]$$

הדבר מצמצם את סיבוכיות זמן הריצה הכוללת ל- $O(n^2)$ בלבד!

מימוש האלגוריתם:

```
def coins_game_dp(V):
    n = len(V)
    # 1. עיבוד מקדים לסכומי רישות (1-based)
    S = [0] * (n + 1)
    for t in range(1, n + 1):
        S[t] = S[t - 1] + V[t - 1]

    # 2. DP-מילודי טבלת ה
    p = [[0] * n for _ in range(n)]

    # מקרי בסיס: אלבסון ראשי (שרשראות באורך 1)
    for i in range(n):
```

```

p[i][i] = V[i]

# n מ-2 עד l מילוי הטבלה לפי אורך השרשרת
for l in range(2, n + 1):
    for i in range(0, n - l + 1):
        j = i + l - 1
        # j עד i סכום השטרות בקטע
        sum_val = S[j + 1] - S[i]
        p[i][j] = sum_val - min(p[i + 1][j], p[i][j - 1])

return p[0][n - 1]

```

סיבוכיות זמן: $O(n^2)$ - שתי לולאות (אורך השרשרת ואינדקס ההתחלה). סיבוכיות מקום: $O(n^2)$ לשמירת הטבלה (ניתן לצמצום ל- $O(n)$ במידה ולא נדרש שחזור).